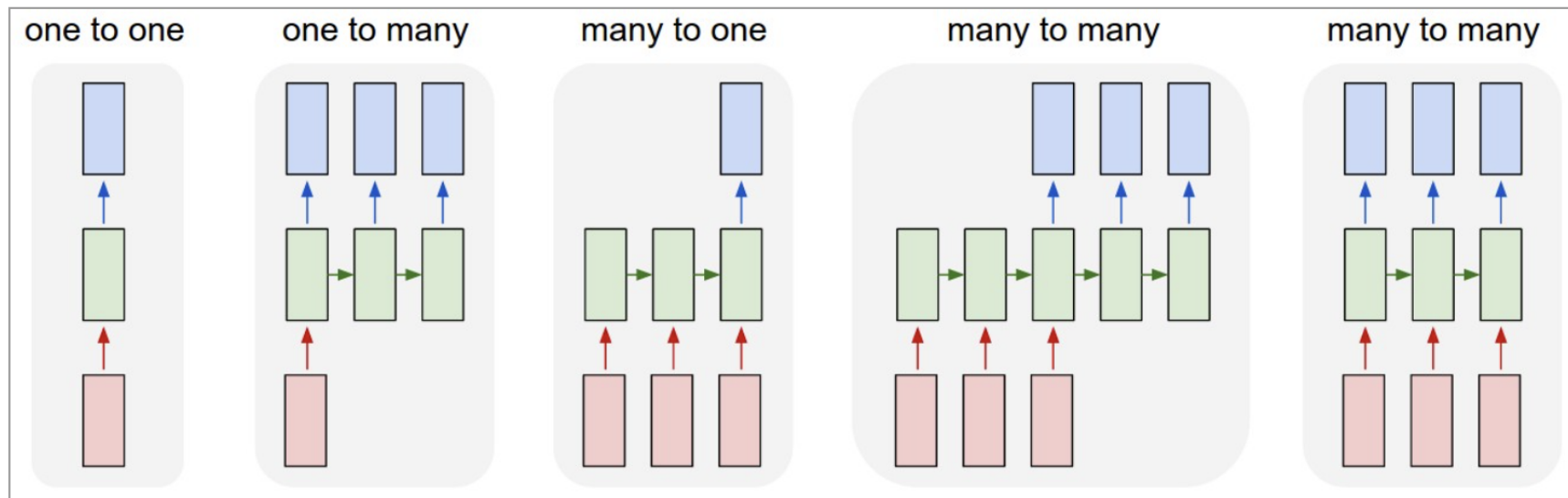


Introduction to Deep Learning

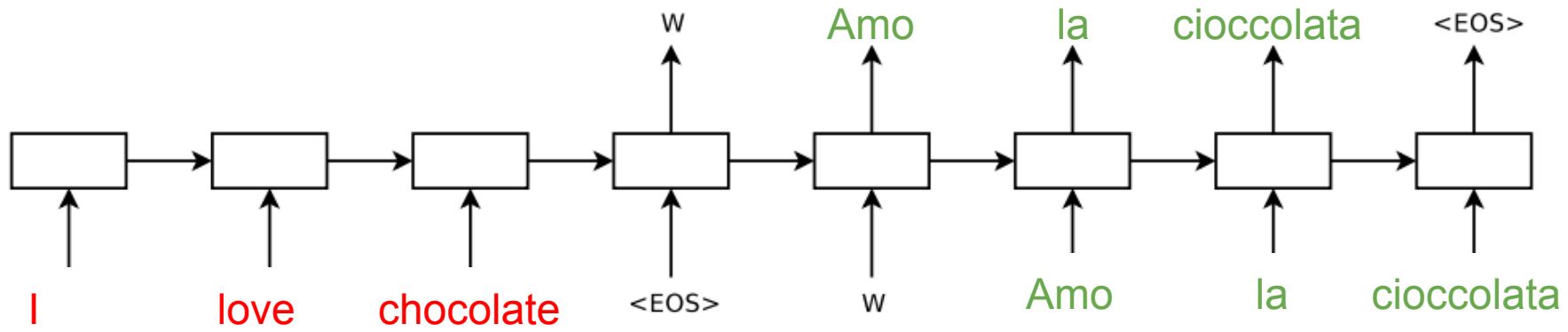
Caltech CS/EE/CMS 148a - Winter 2025

Lecture 11 - Self-Attention

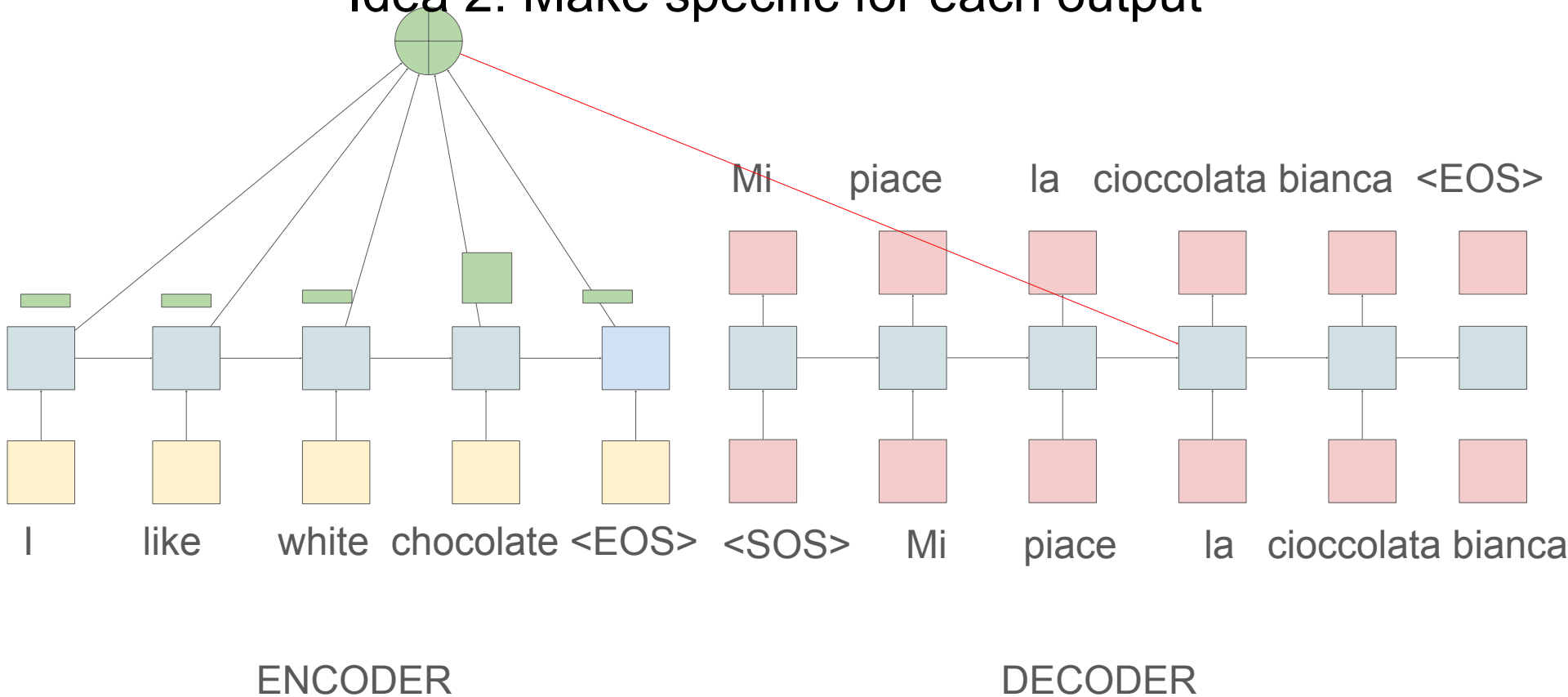
Recap

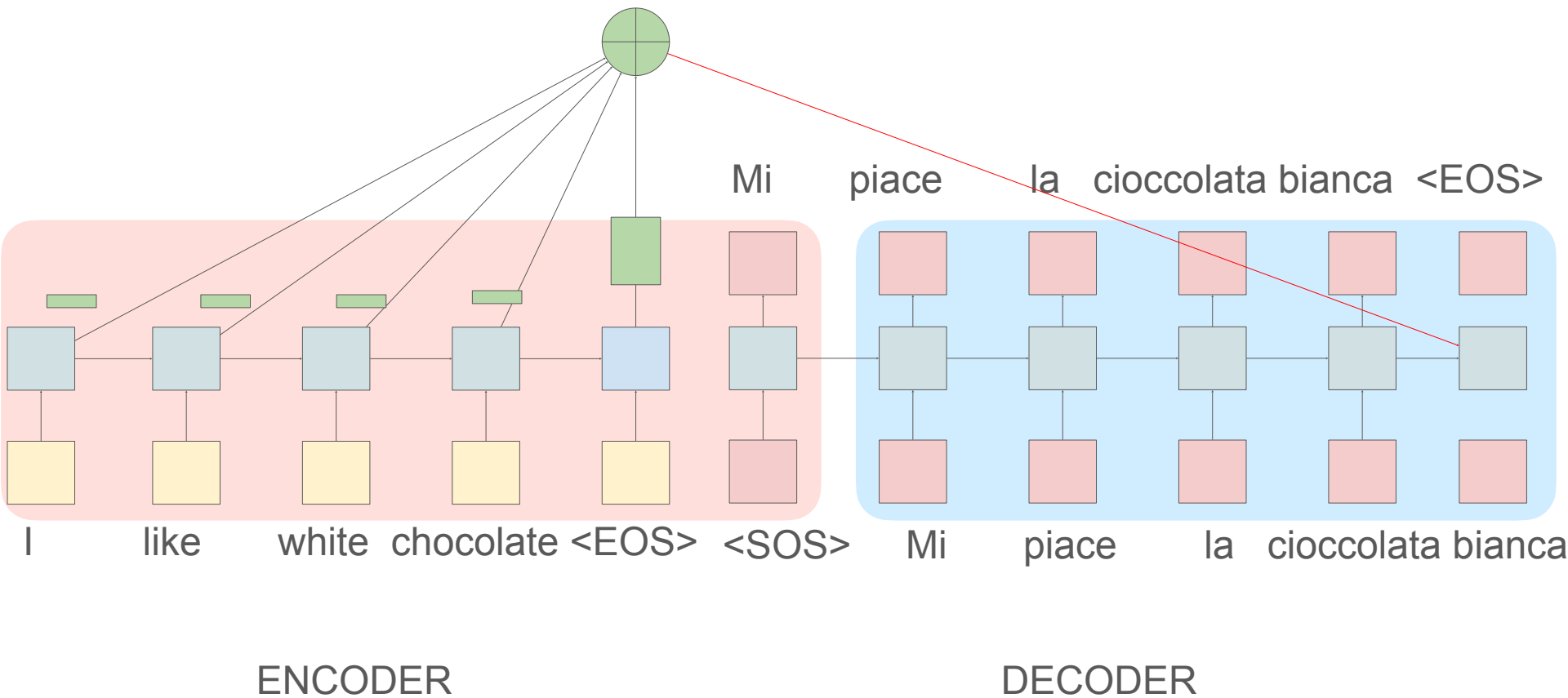


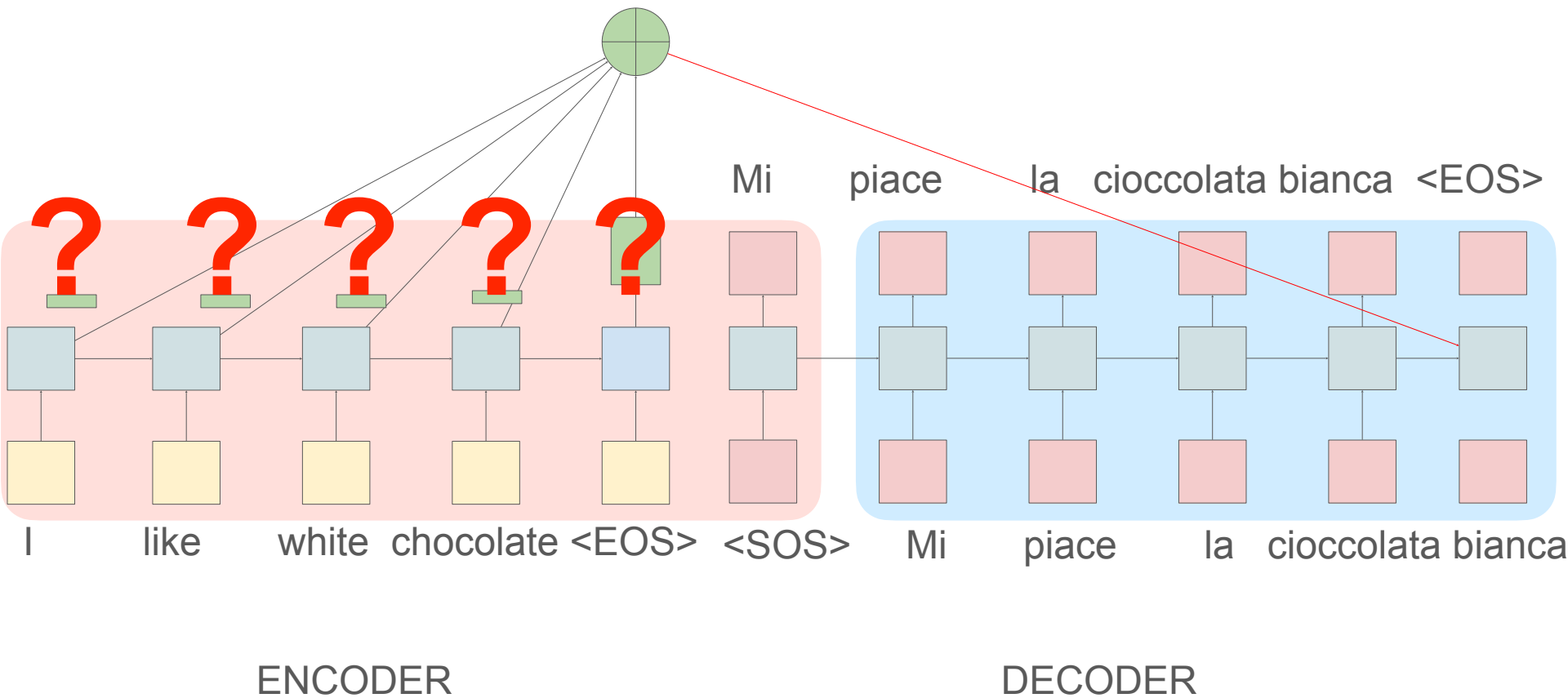
Sequence to Sequence Learning with Neural Networks (Sutskever et al, 2014)



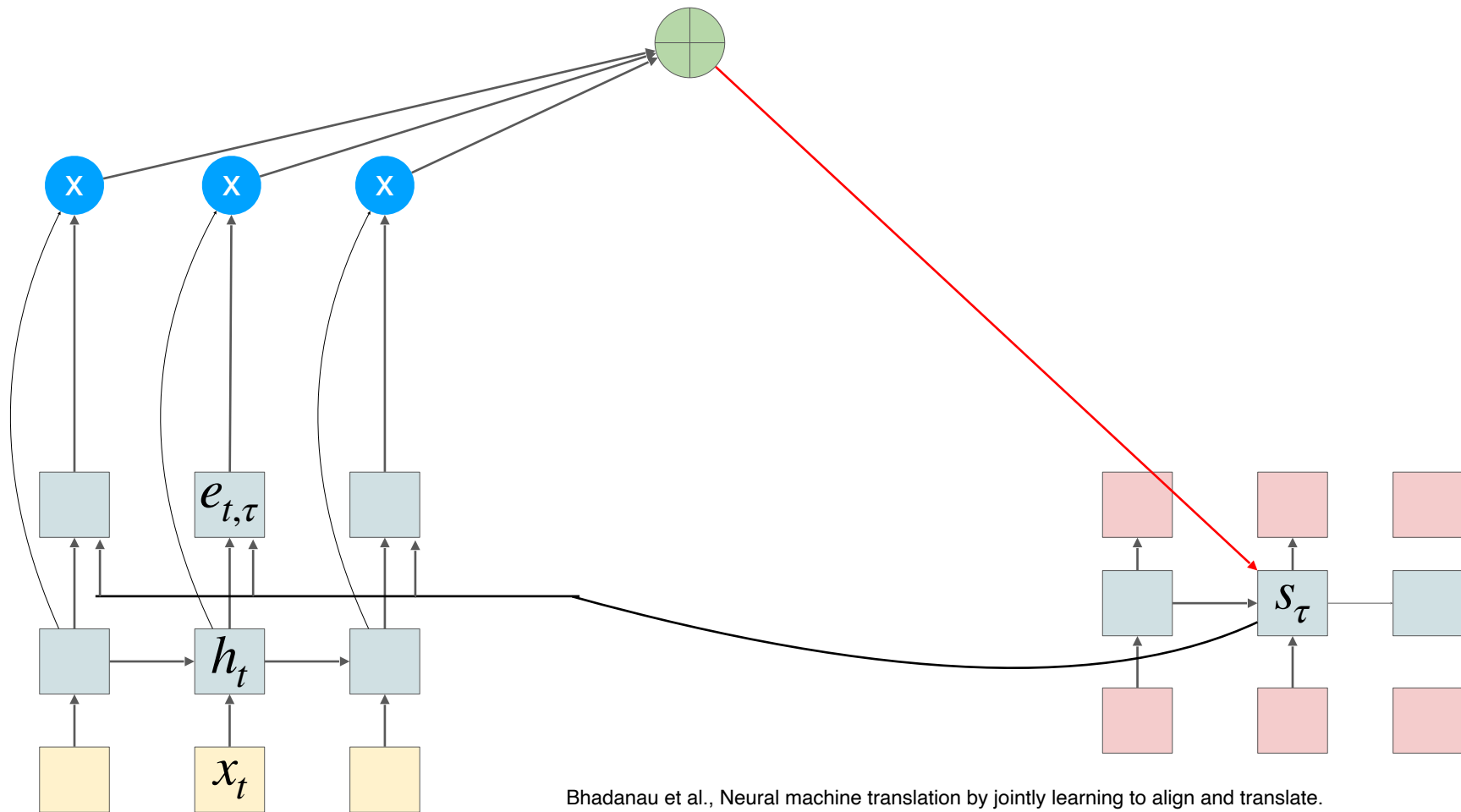
Idea 2: Make specific for each output



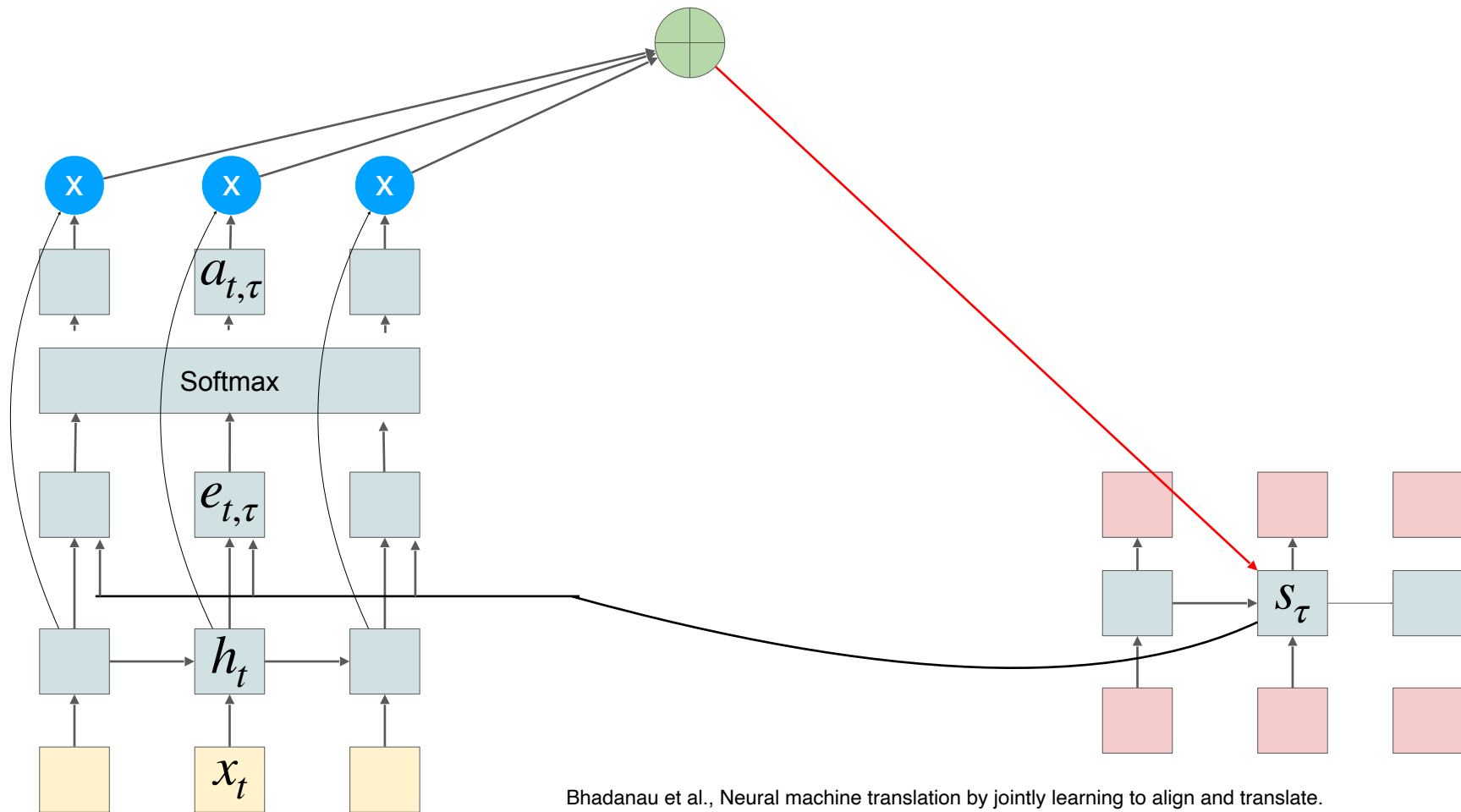




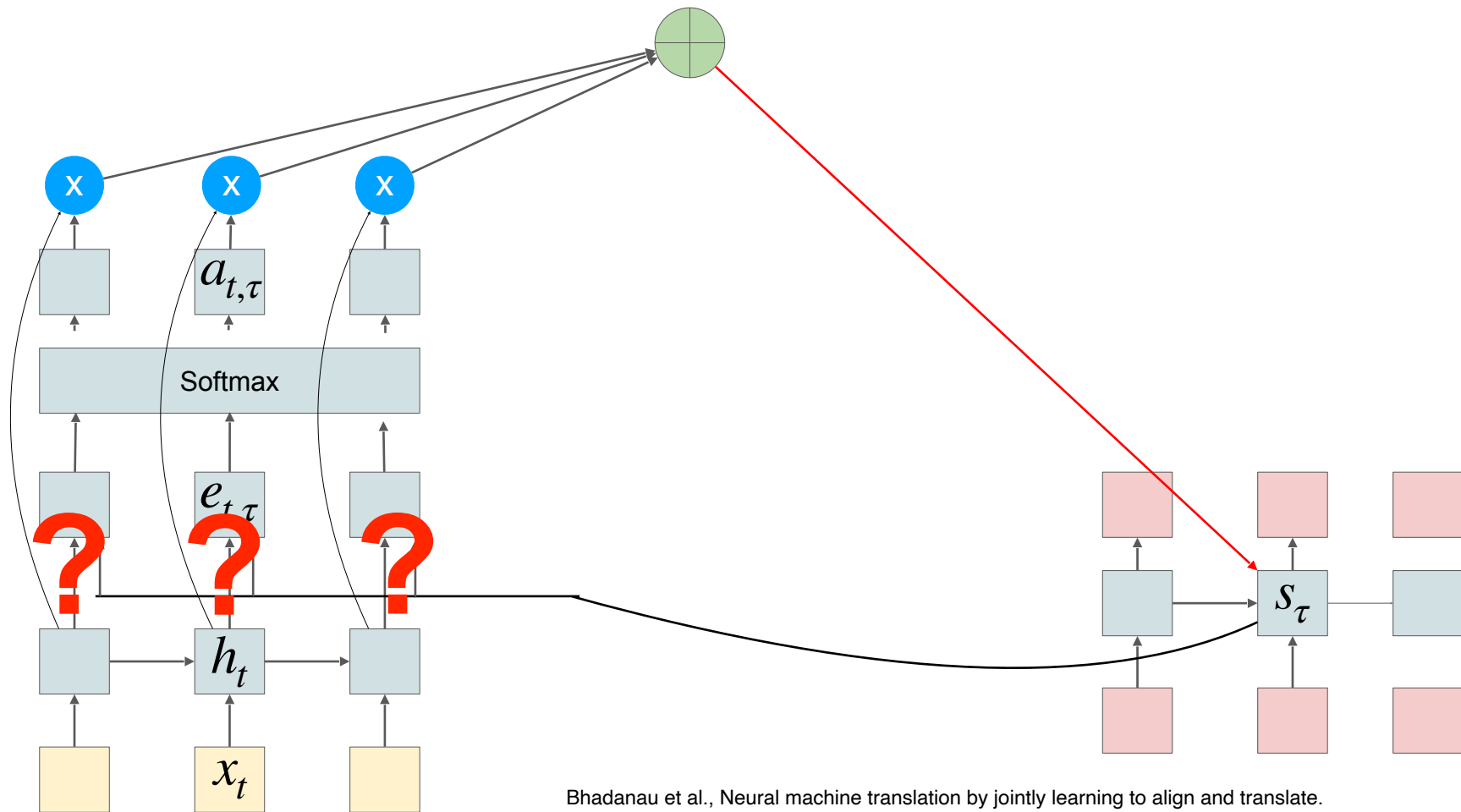
Idea 2: Weights **specific** for each input



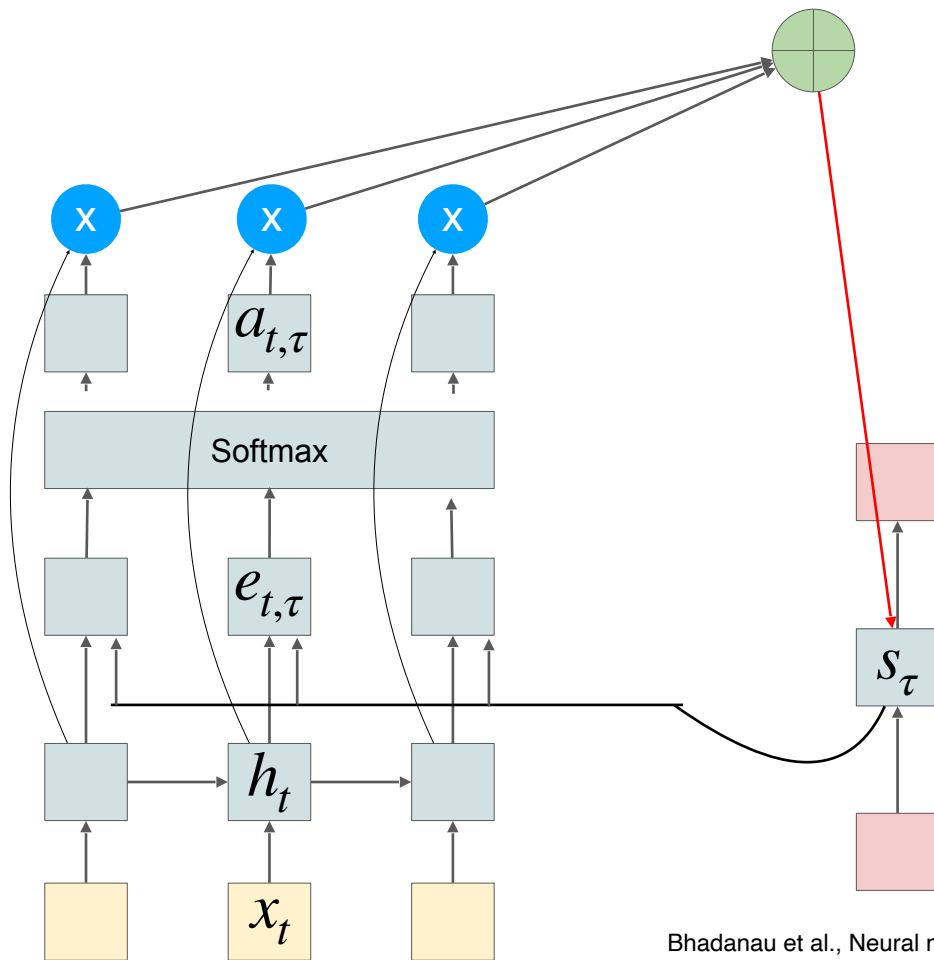
Computing the weights?



Computing the weights?

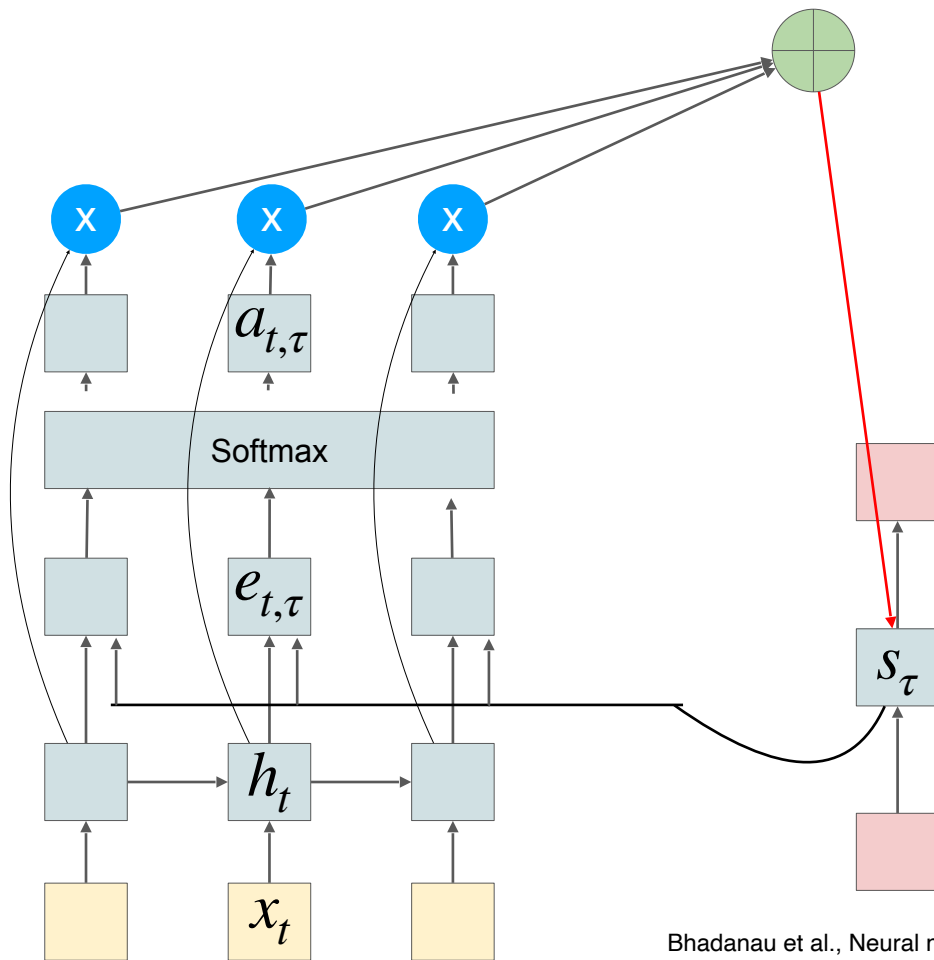


Computing the weights

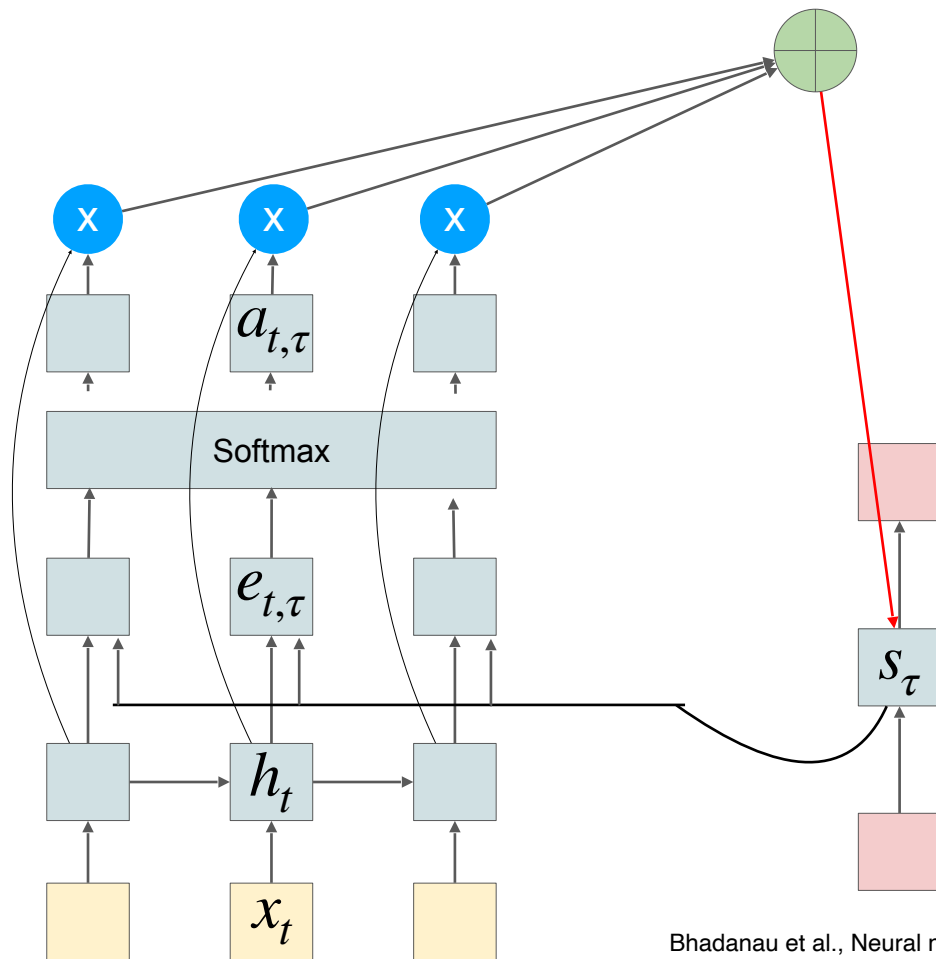


Computing the weights

$$e_{t,\tau} = \text{similarity}(h_t, s_\tau)$$



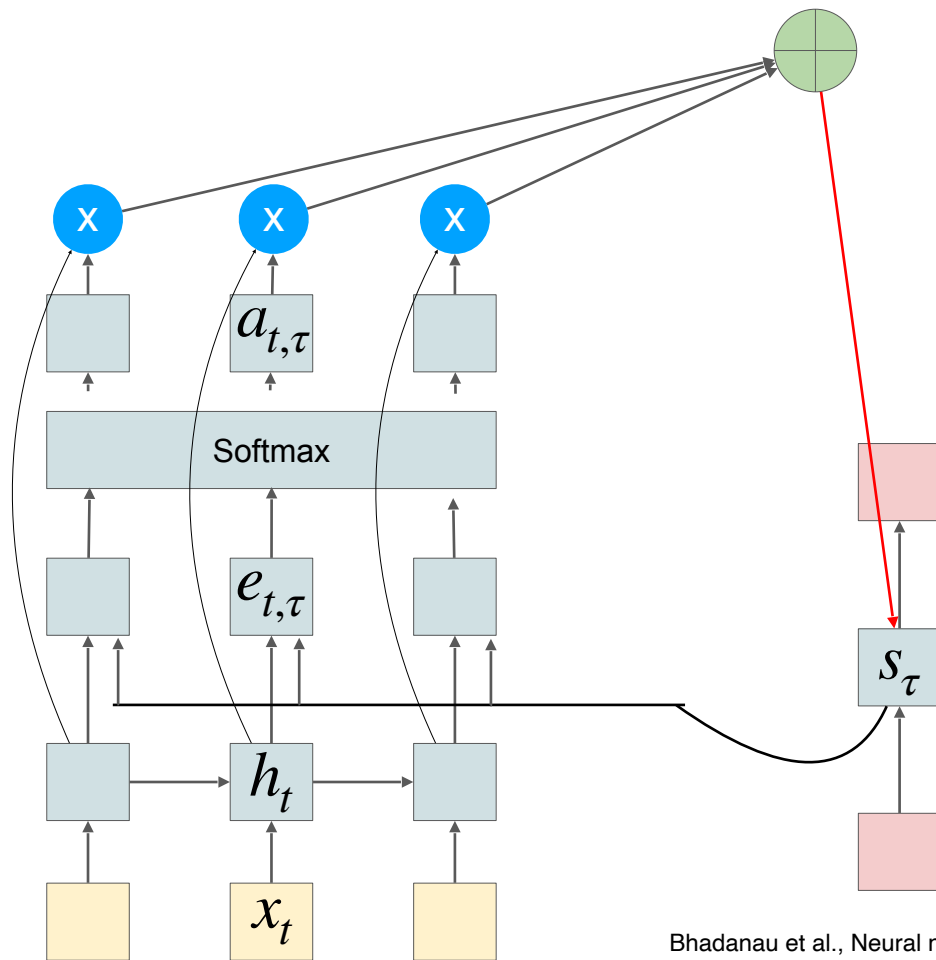
Computing the weights



$$e_{t,\tau} = \text{similarity}(h_t, s_\tau)$$

$$e_{t,\tau} = \langle h_t, s_\tau \rangle$$

Computing the weights



$$e_{t,\tau} = \text{similarity}(h_t, s_\tau)$$

$$e_{t,\tau} = \langle h_t, s_\tau \rangle$$

$$e_{t,\tau} = \frac{\langle h_t, s_\tau \rangle}{\sqrt{D}}$$

Attention: Query, Key, Value

Data

Joseph, 11

Wendy, 12

Nellie, 7

Mary, 13

Theodore, 7

Attention: Query, Key, Value

Data

Joseph, 11

Wendy, 12

Nellie, 7

Mary, 13

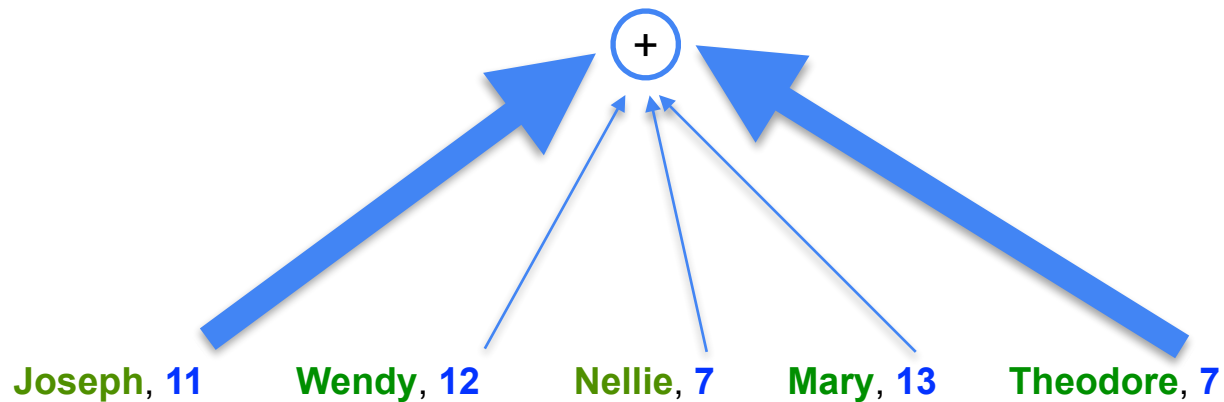
Theodore, 7

Boys' age?

Attention: Query, Key, Value

Data

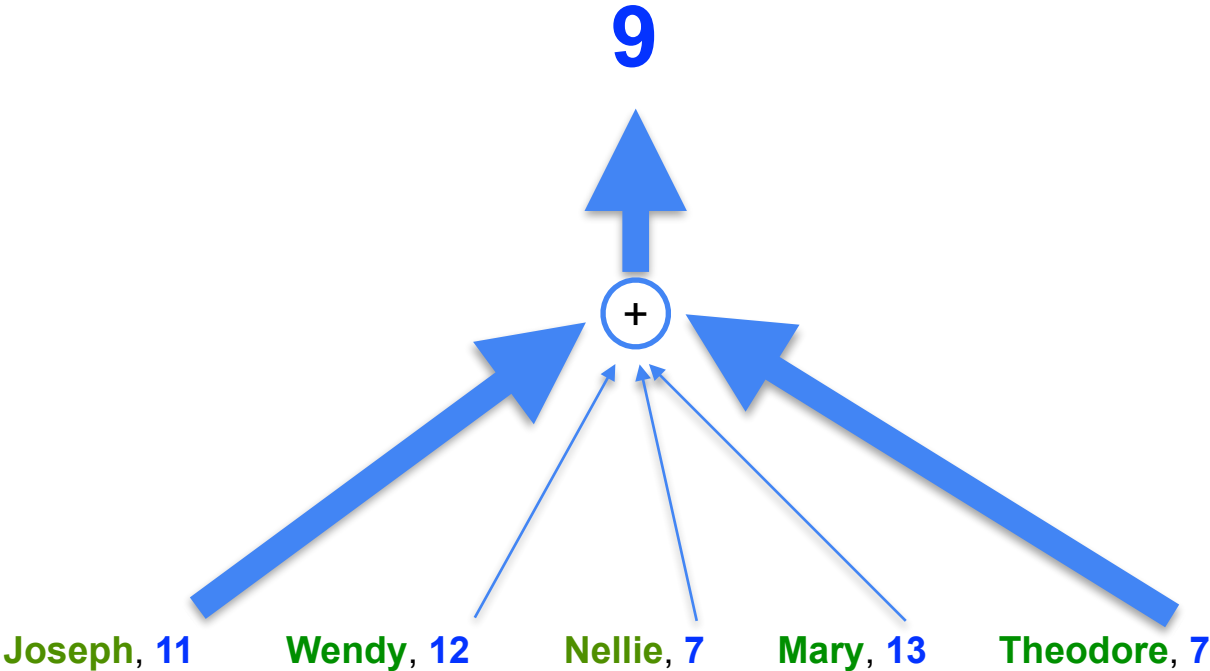
Boys' age?



Attention: Query, Key, Value

Data

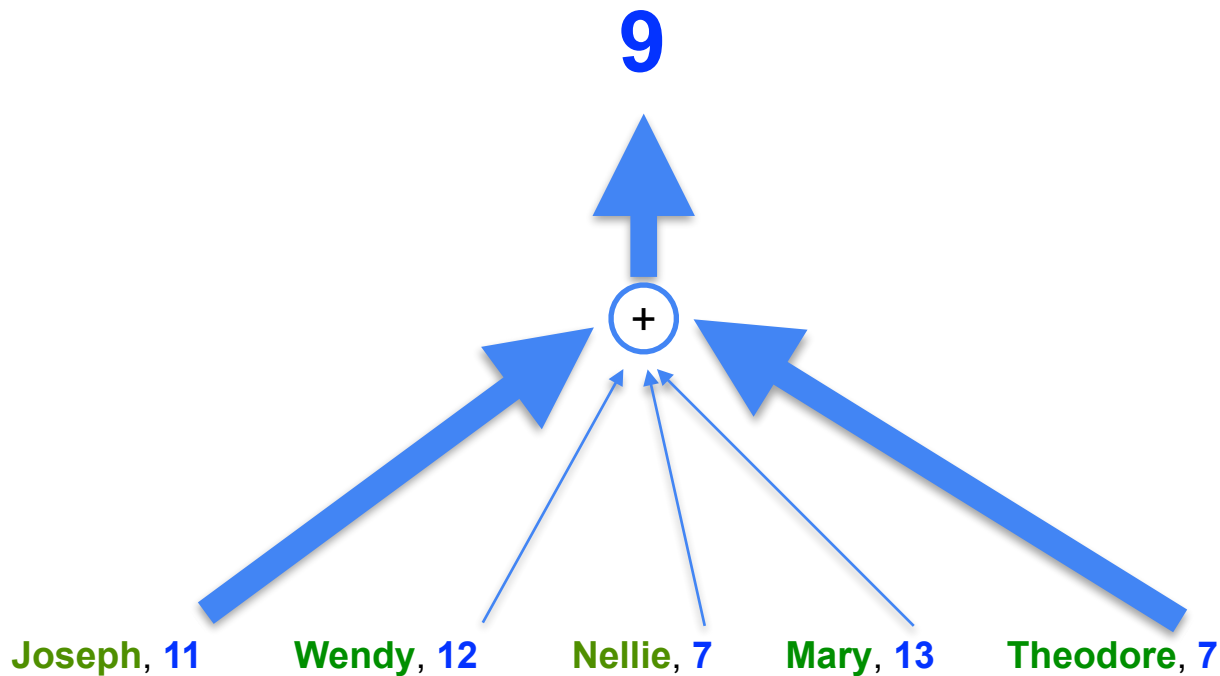
Boys' age?



Attention: Query, Key, Value

Data

Boys' age?



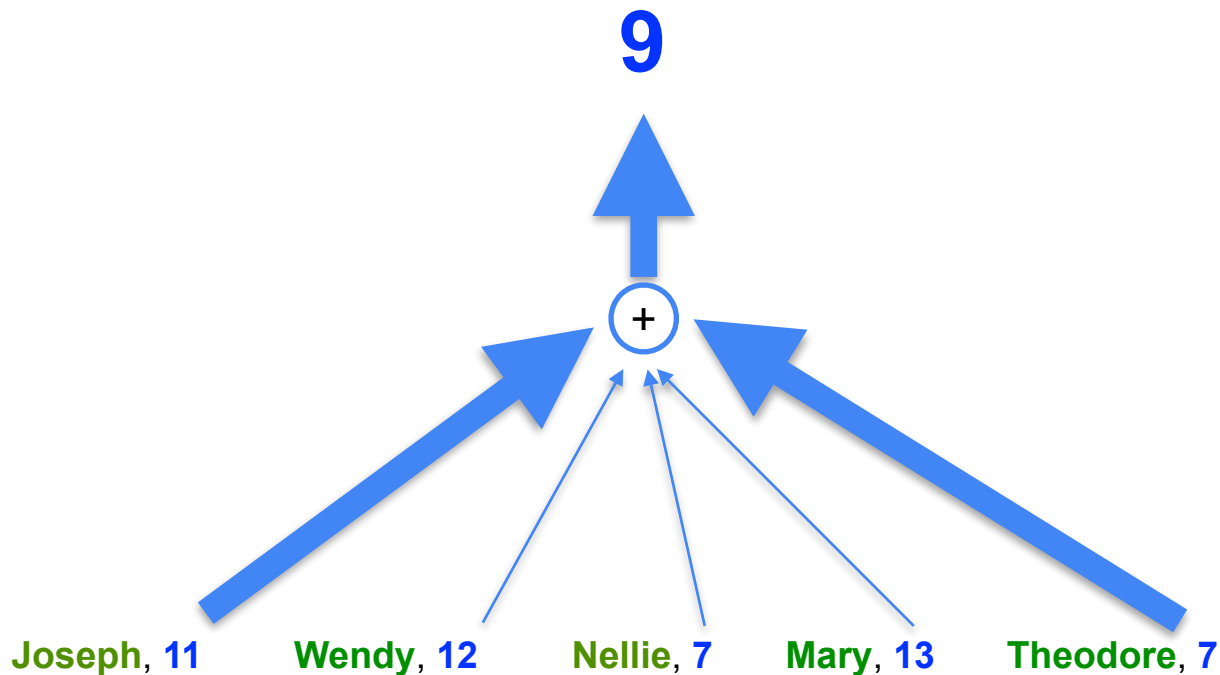
Key, Value

Attention: Query, Key, Value

Data

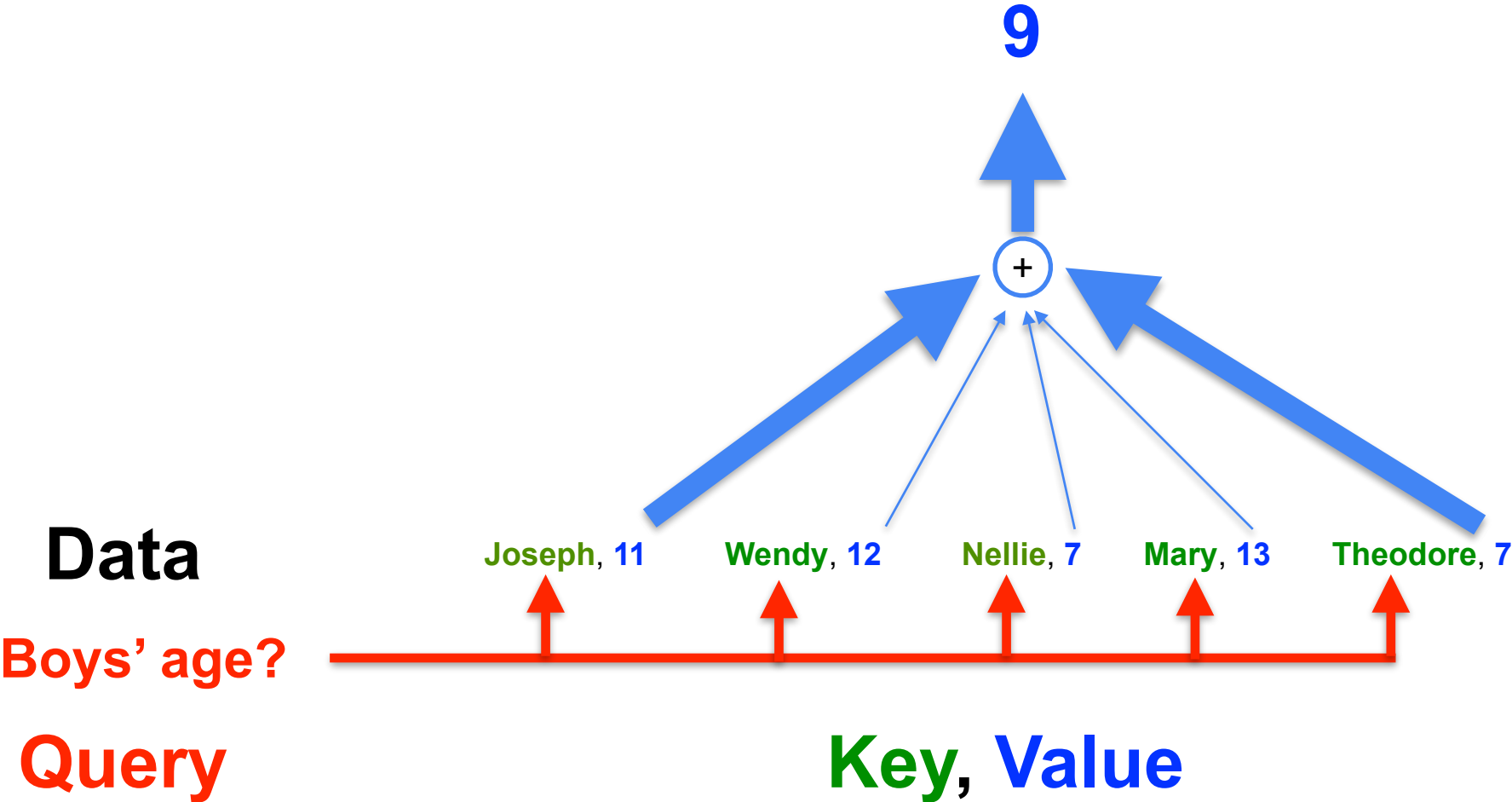
Boys' age?

Query

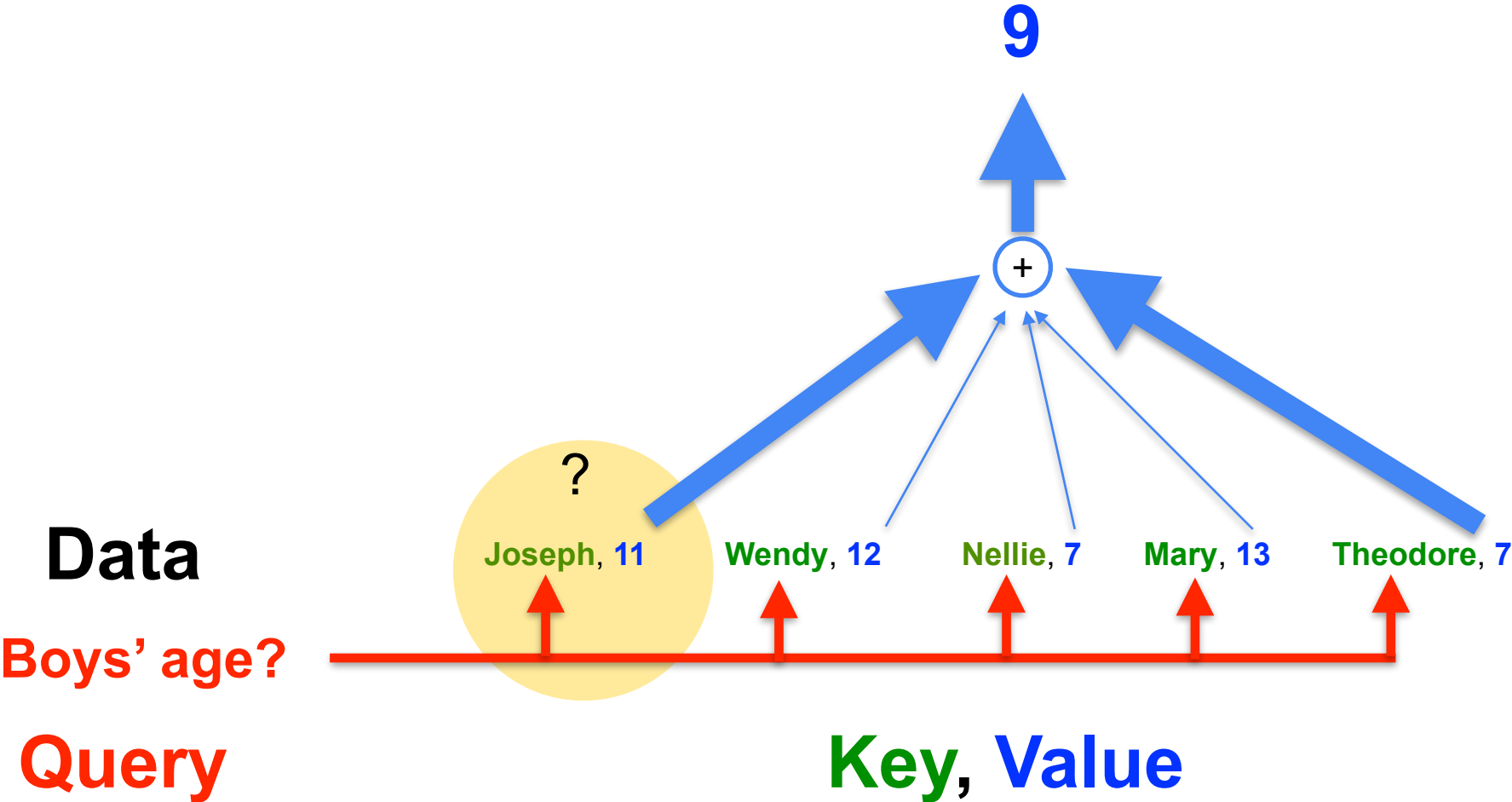


Key, Value

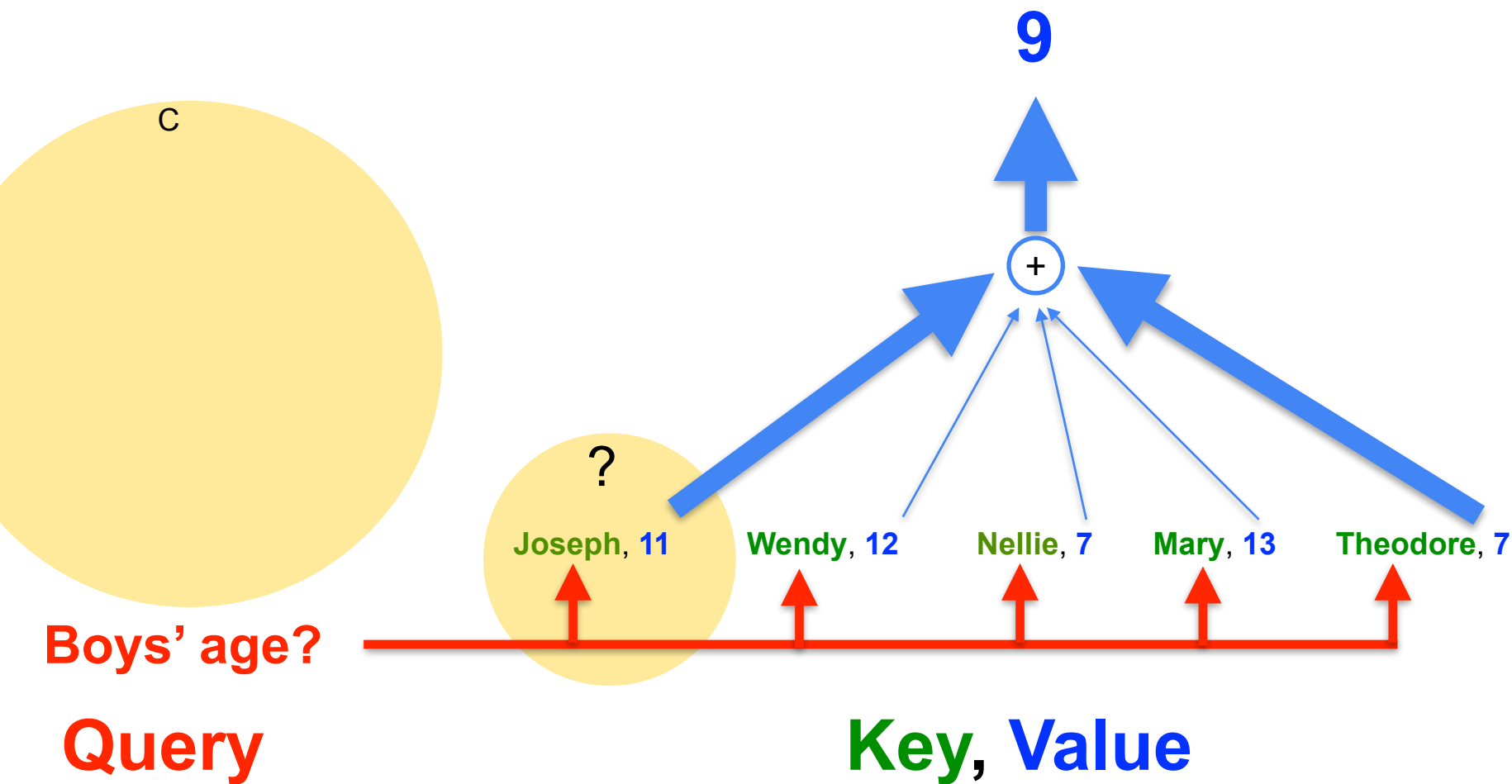
Attention: Query, Key, Value



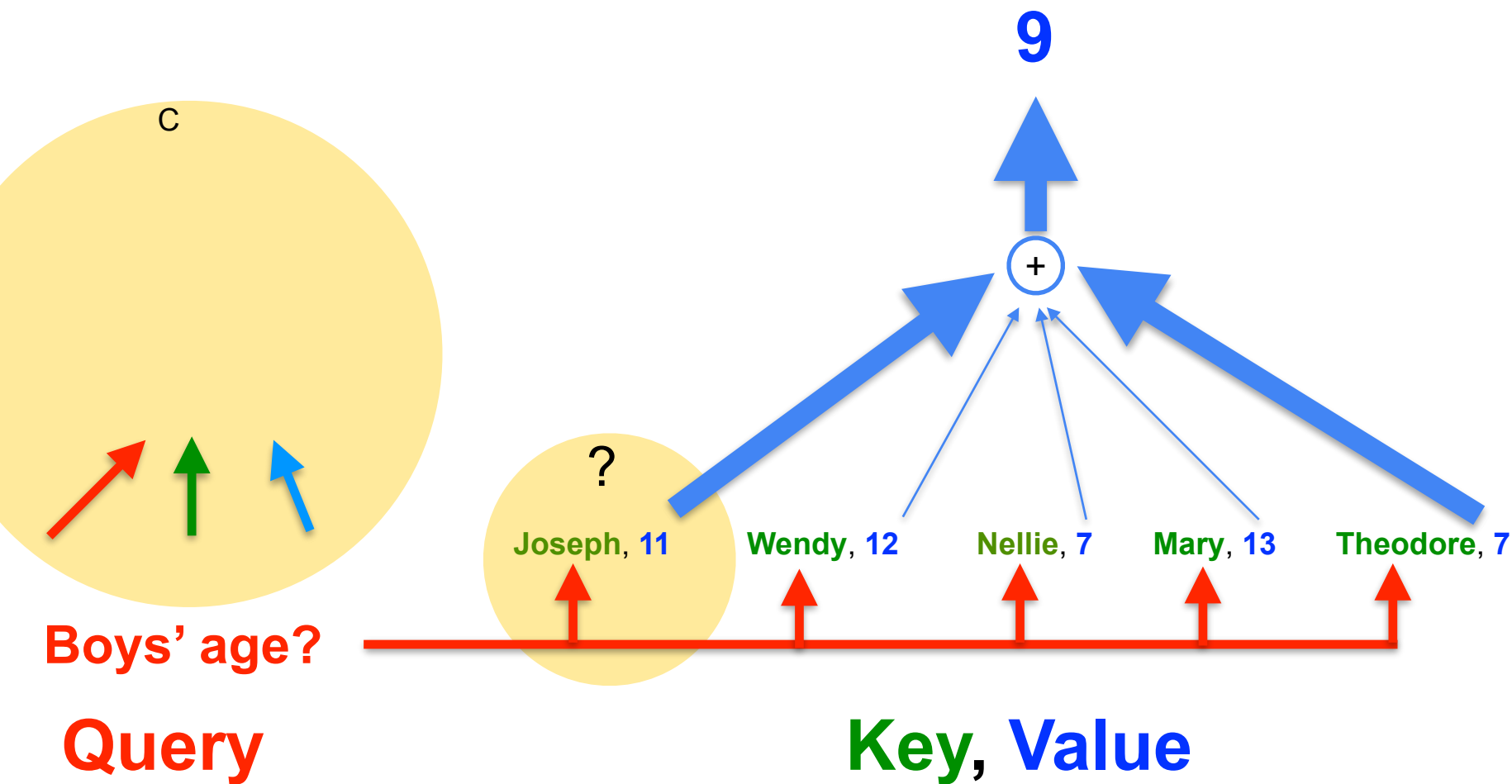
Attention: Query, Key, Value



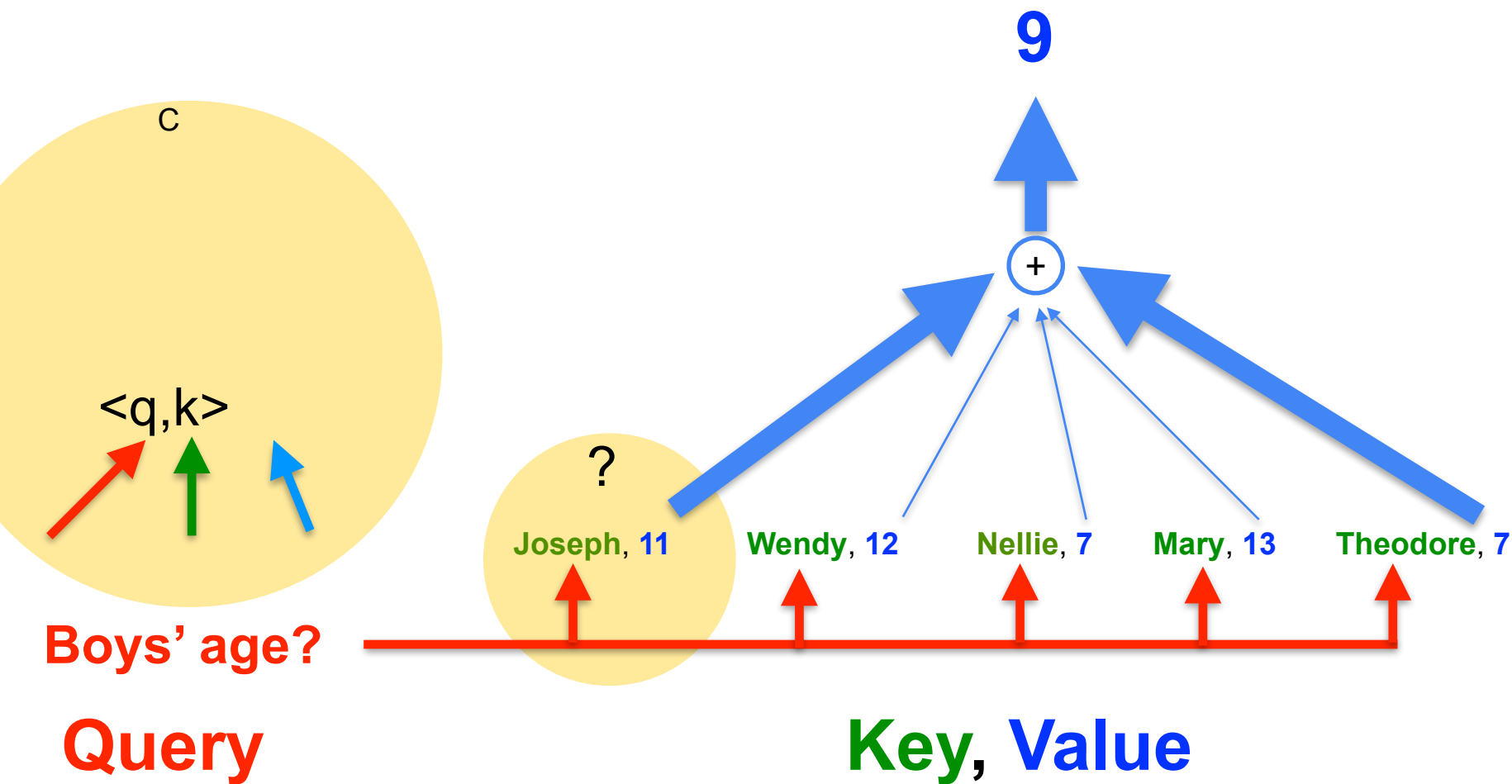
Attention: Query, Key, Value



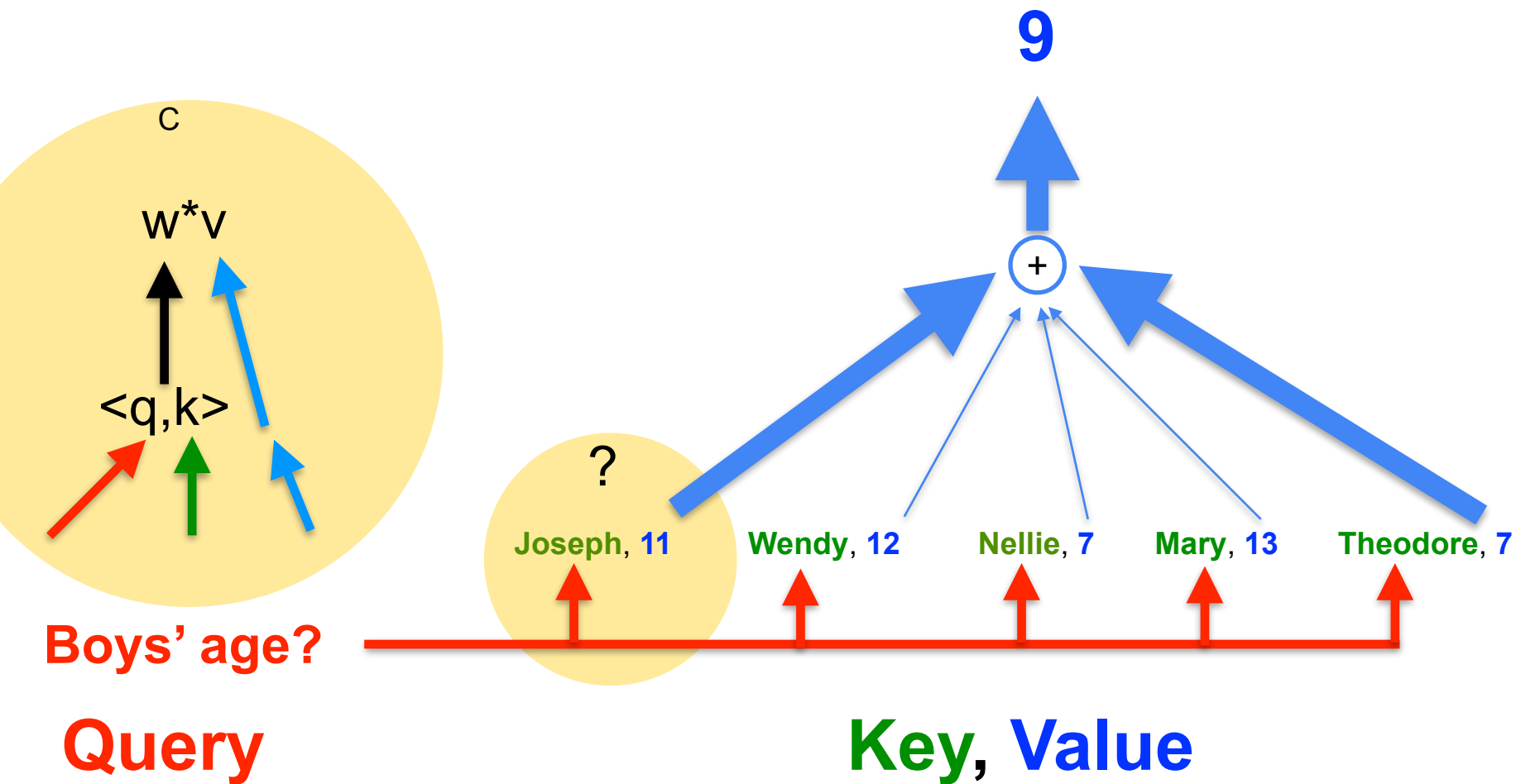
Attention: Query, Key, Value



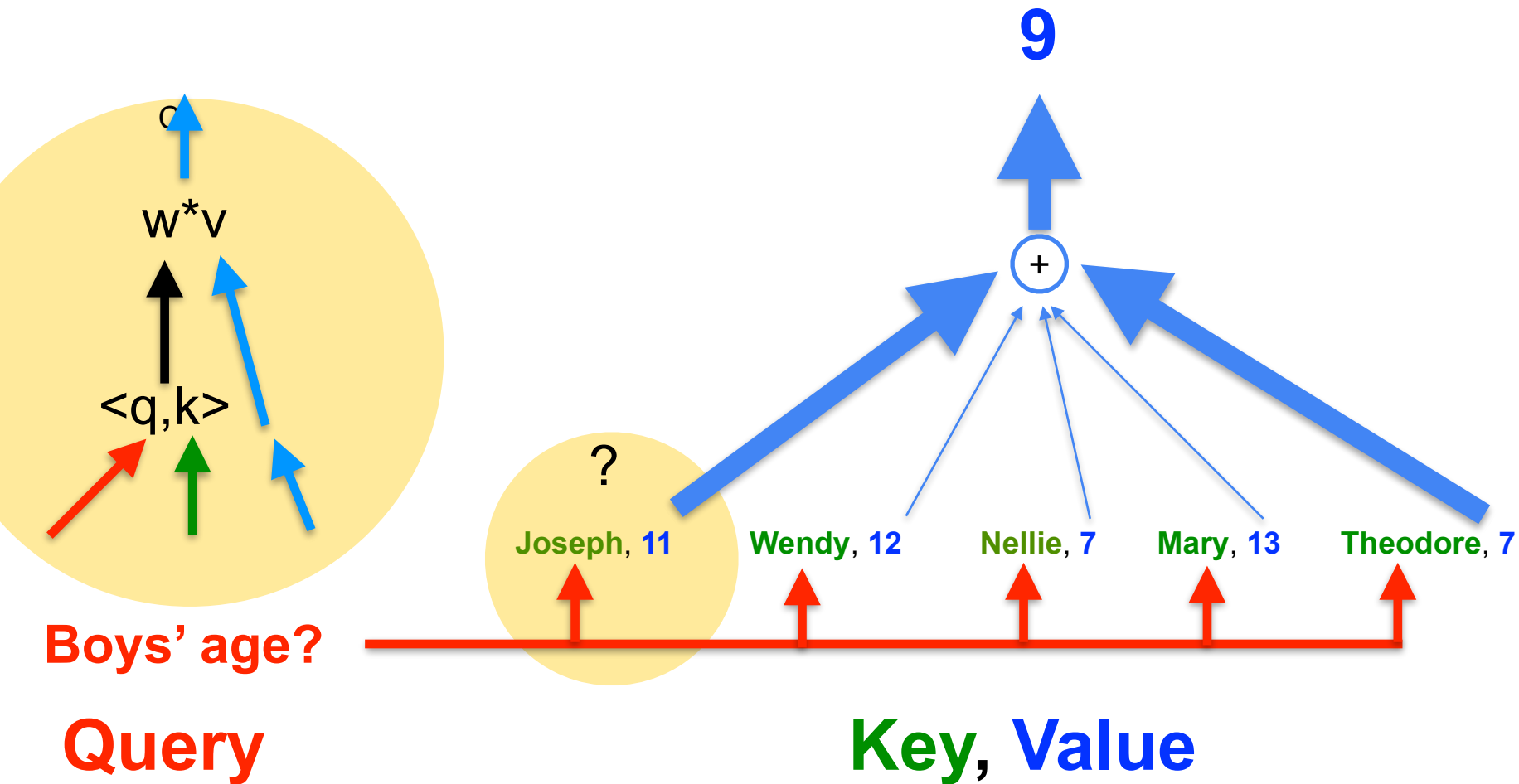
Attention: Query, Key, Value



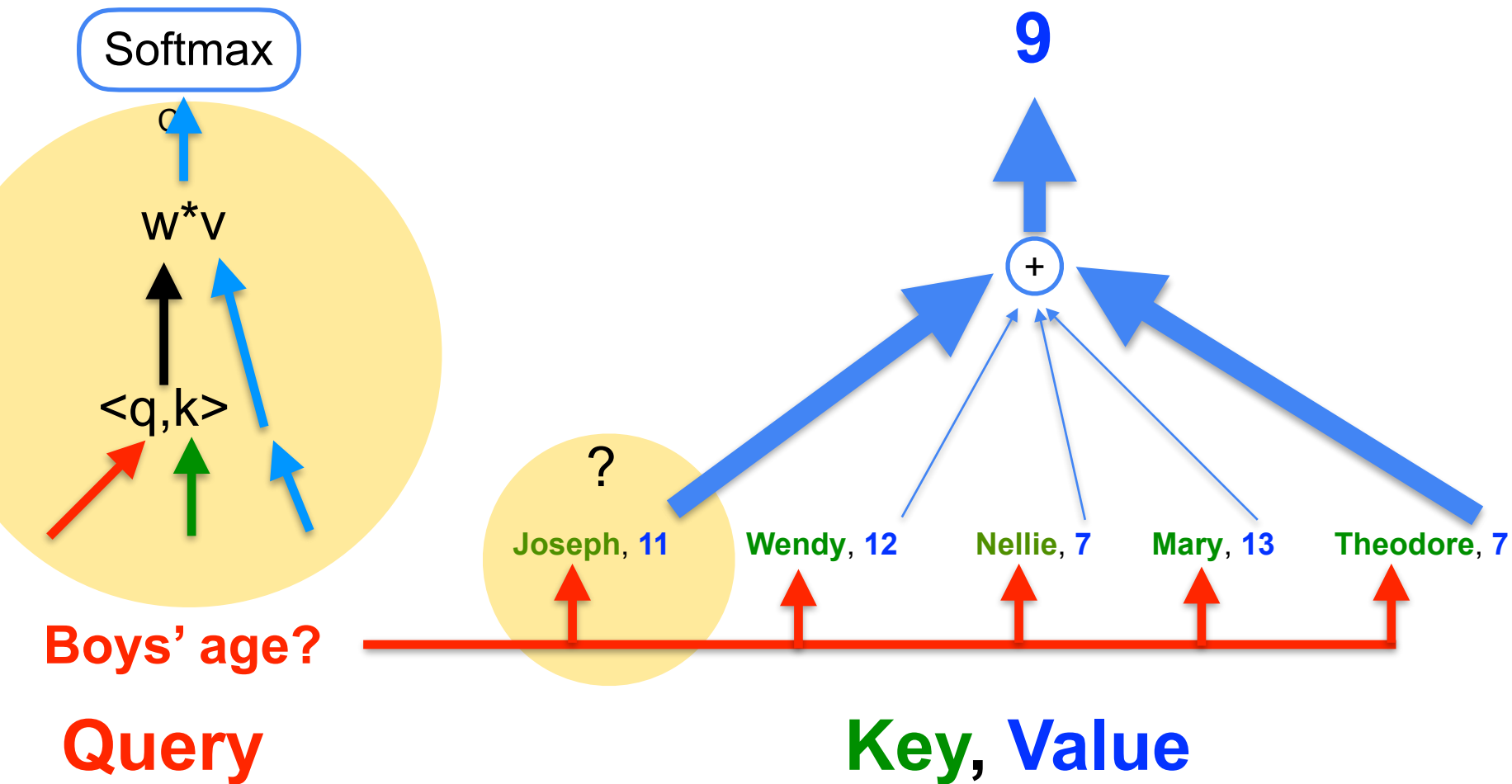
Attention: Query, Key, Value



Attention: Query, Key, Value



Attention: Query, Key, Value



Attention on multiple query vectors

$$Q \in \mathbb{R}^{n \times d}$$

n queries

$$K \in \mathbb{R}^{m \times d}$$

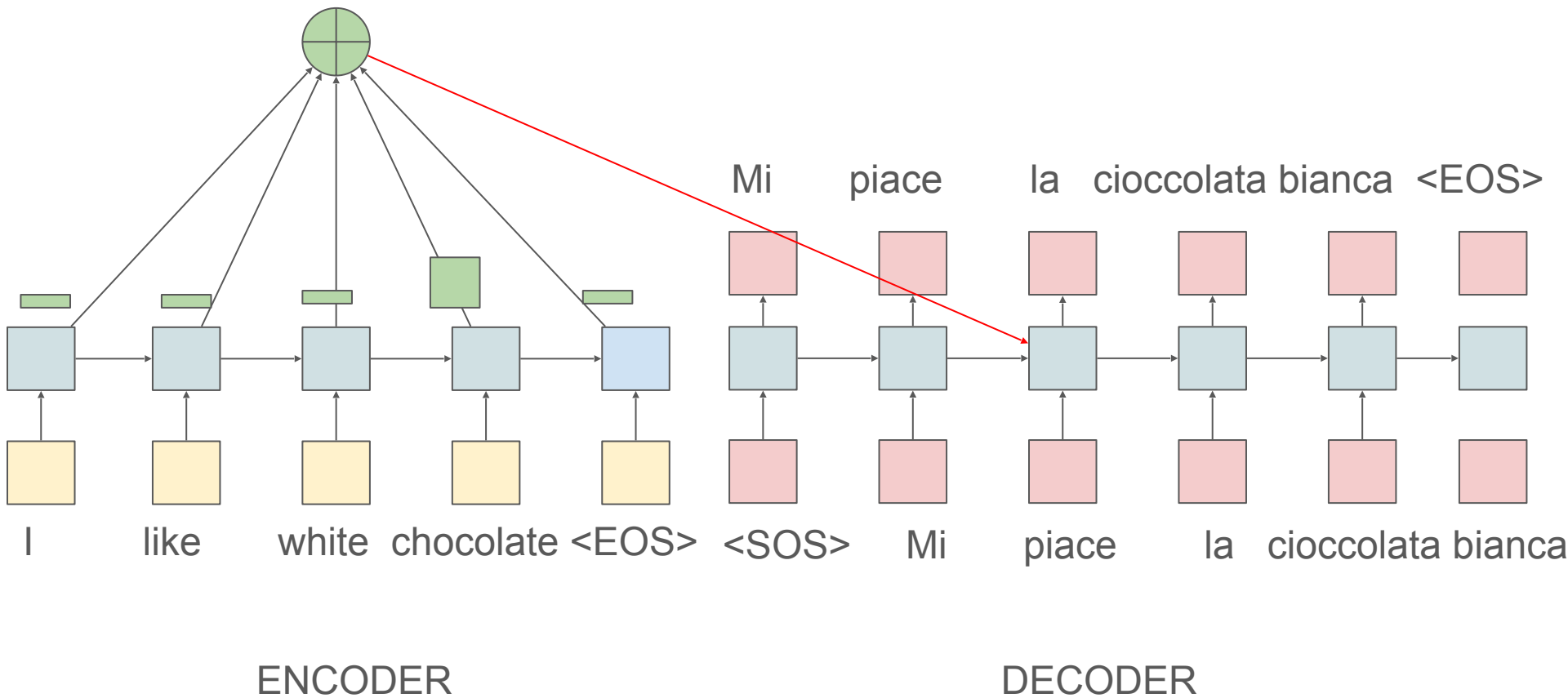
m key-value pairs,
with value of
dimension d

$$V \in \mathbb{R}^{m \times v}$$

$$\text{softmax}\left(\frac{Q^{\top} K}{\sqrt{d}}\right) V \in \mathbb{R}^{n \times v}$$

n attention
vectors of
dimension d

Encoder-decoder with attention



Self Attention

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

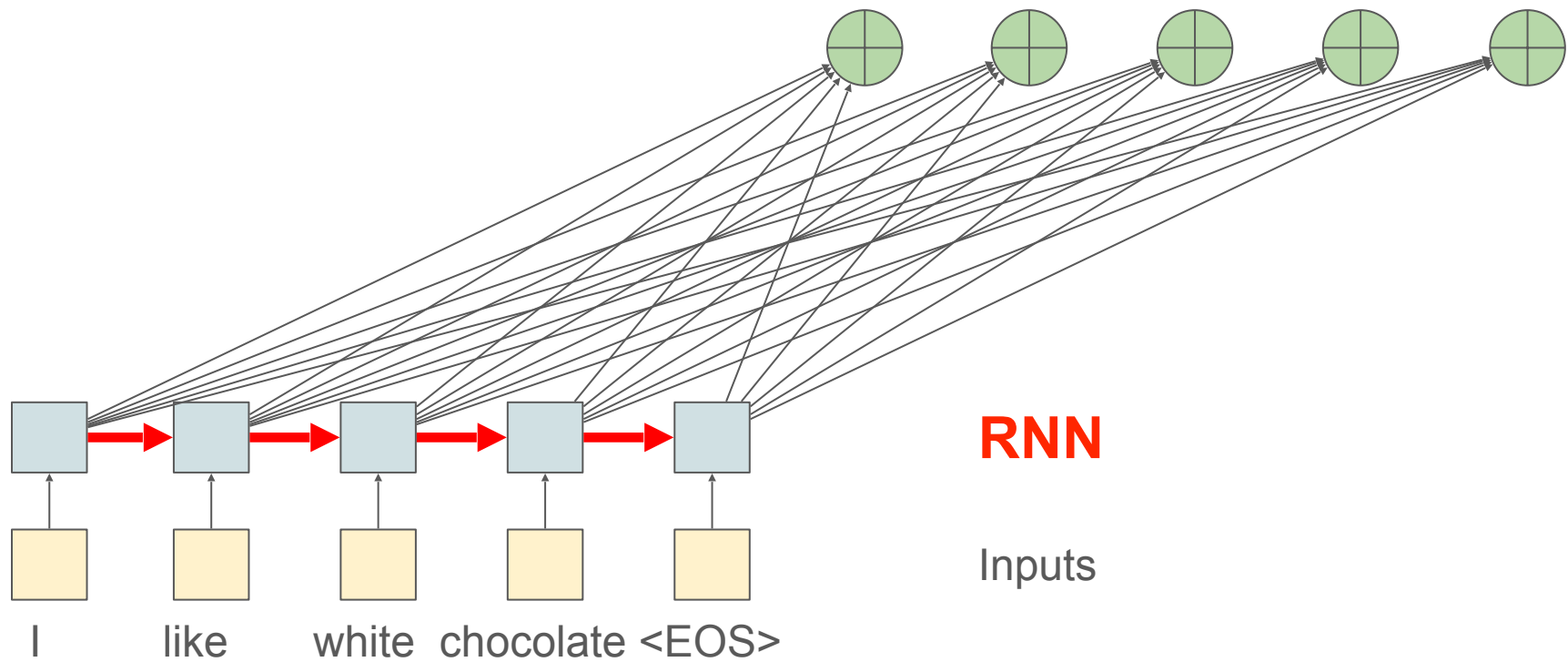
Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

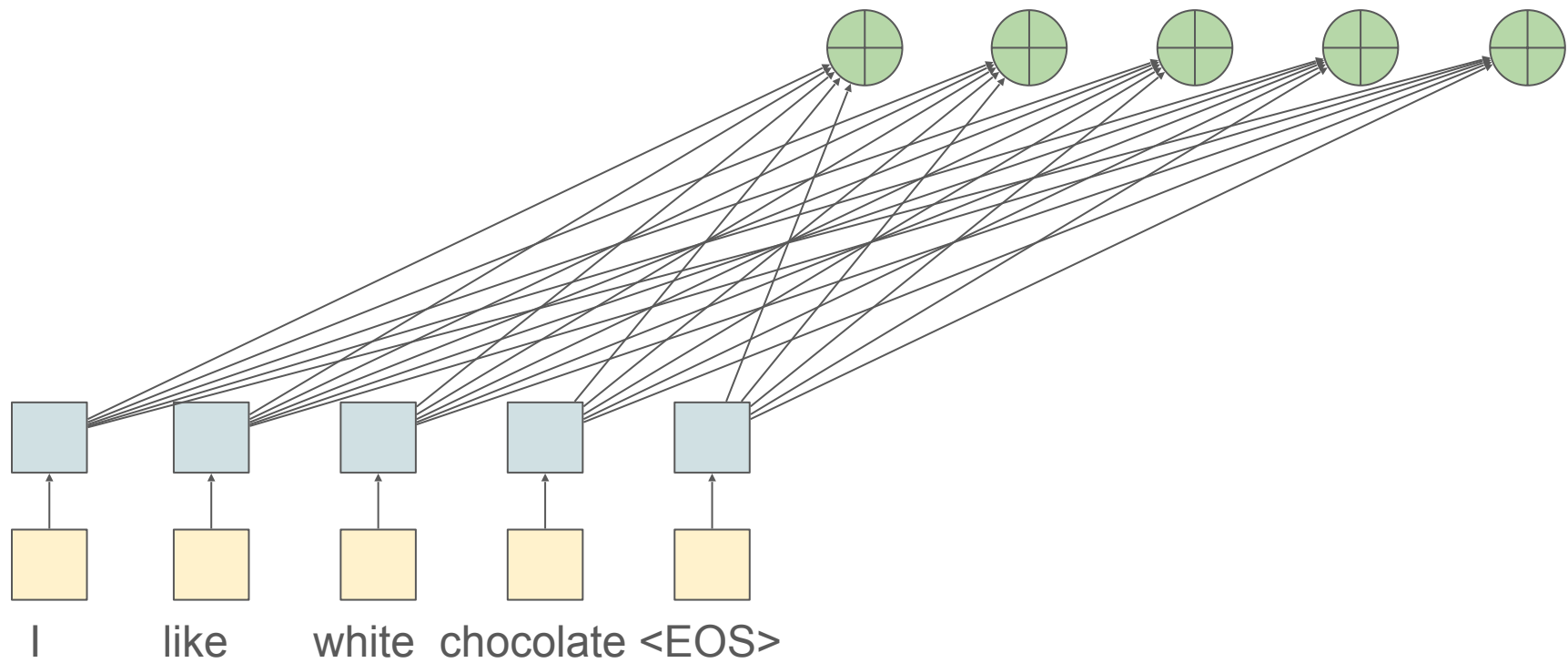
Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

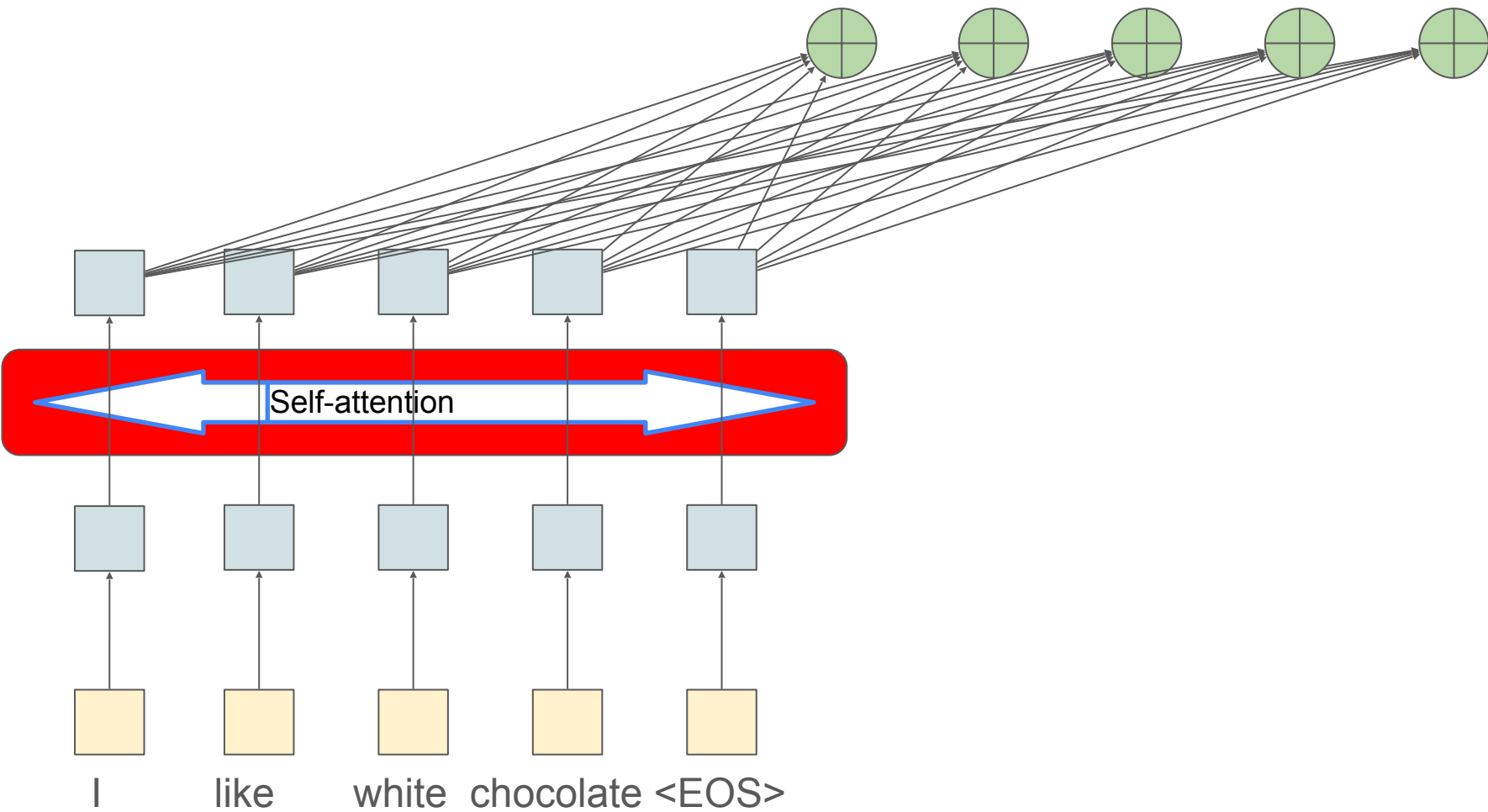
(NIPS 2017)

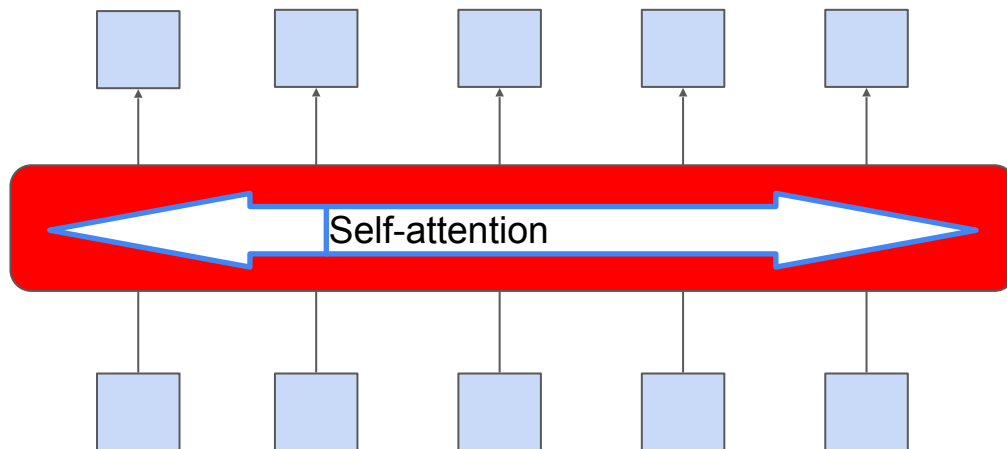


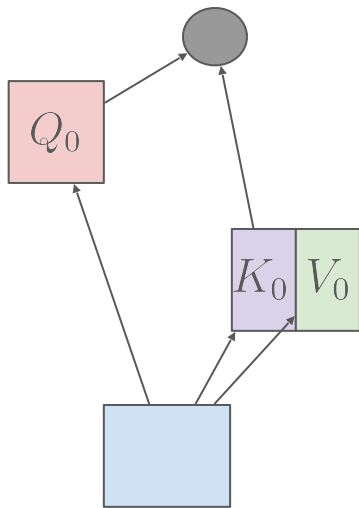
ENCODER

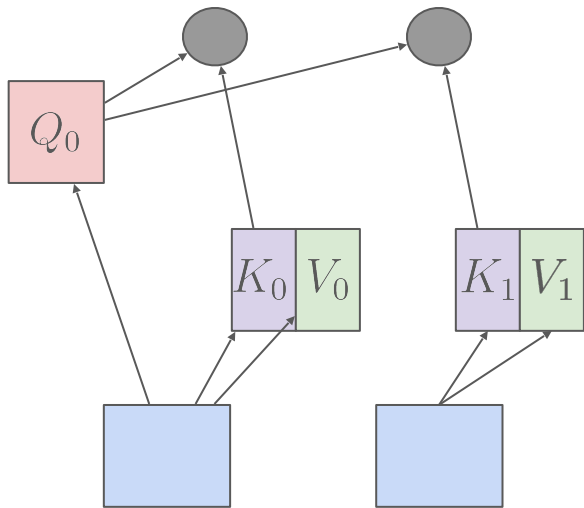


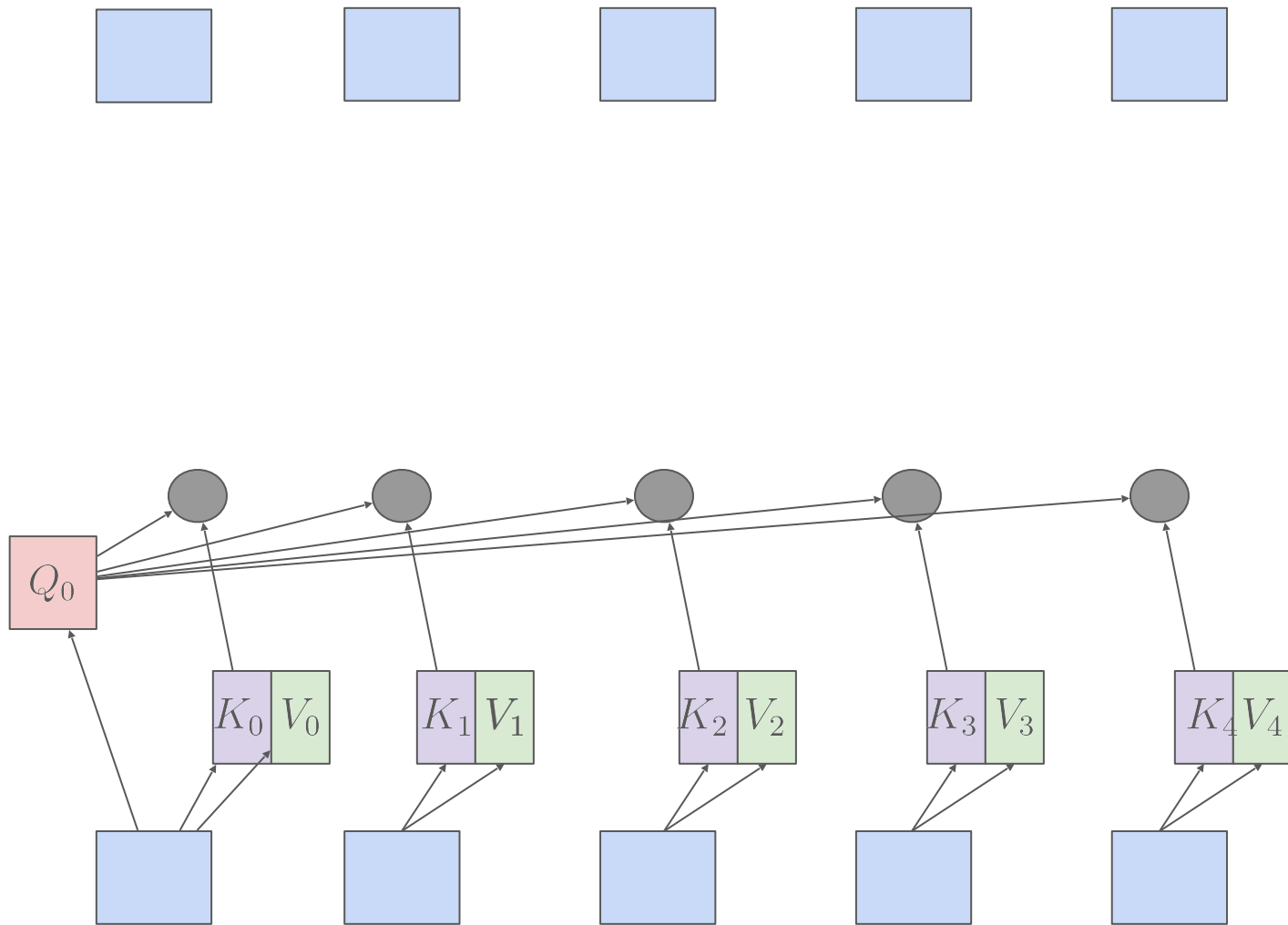
ENCODER

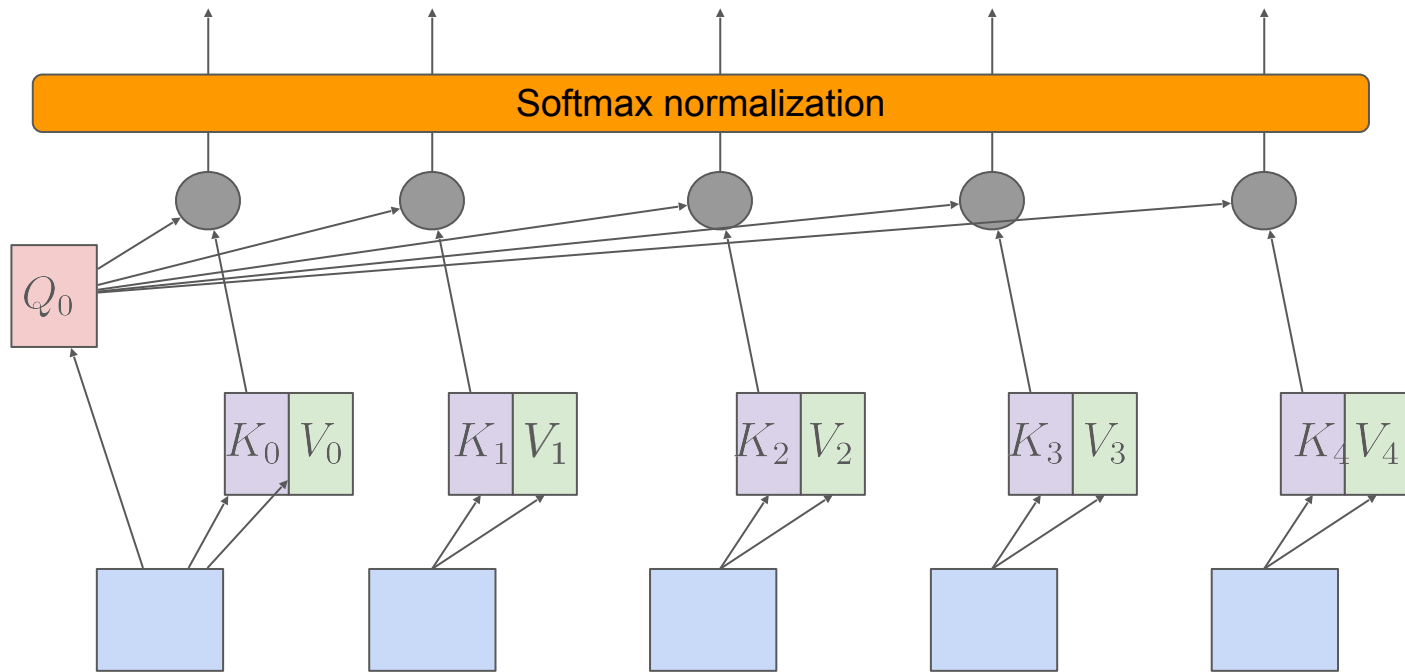


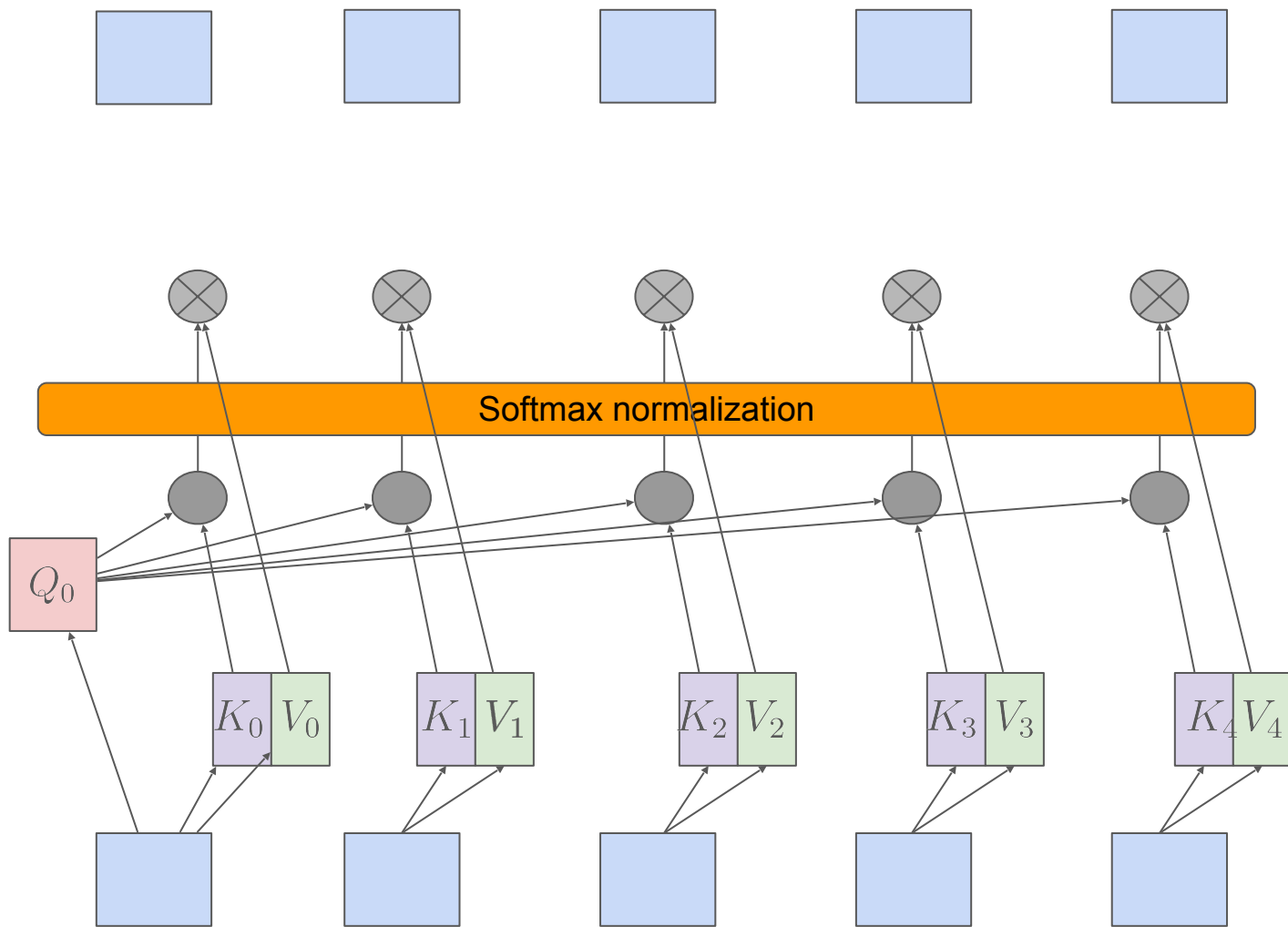


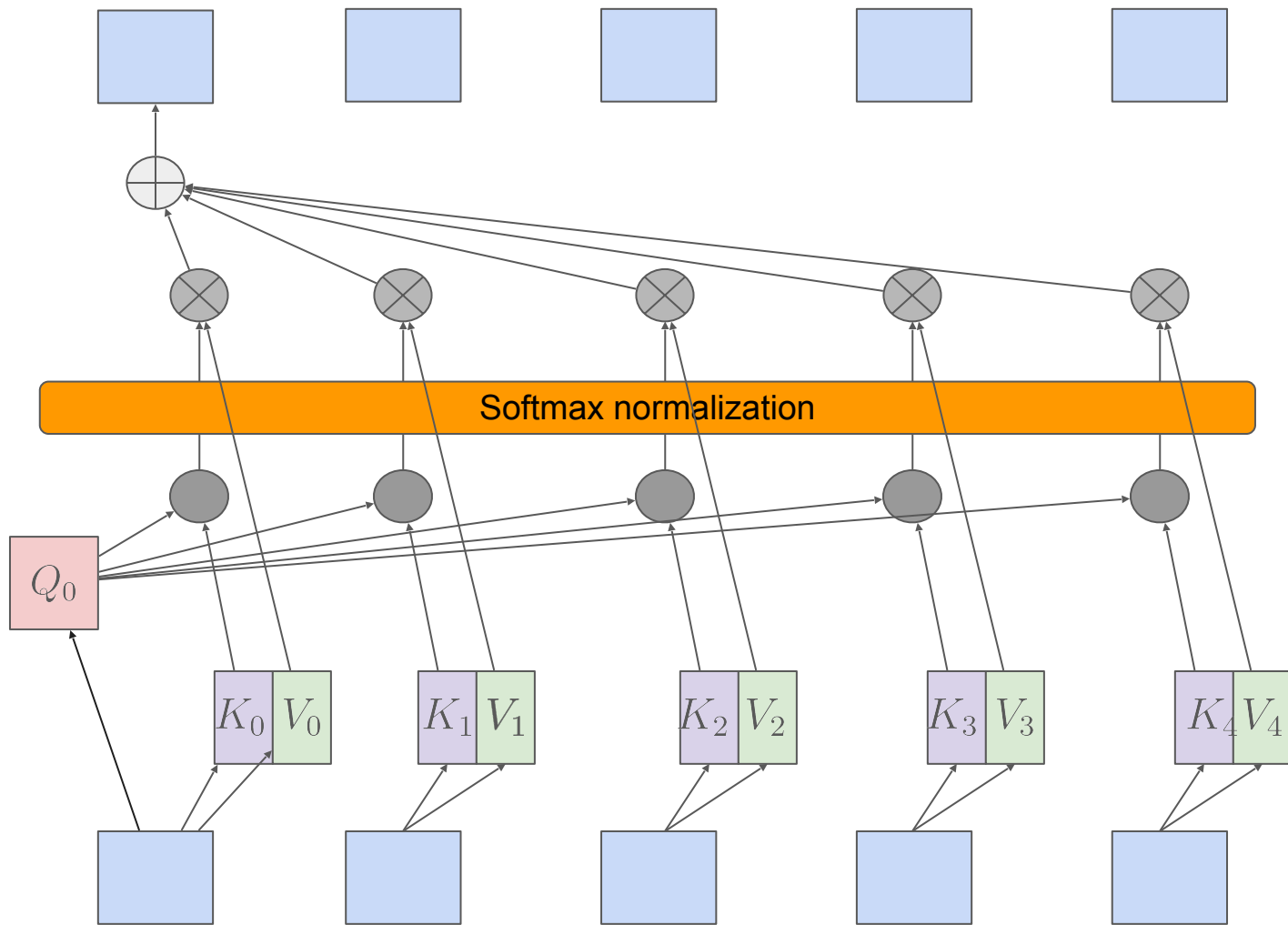


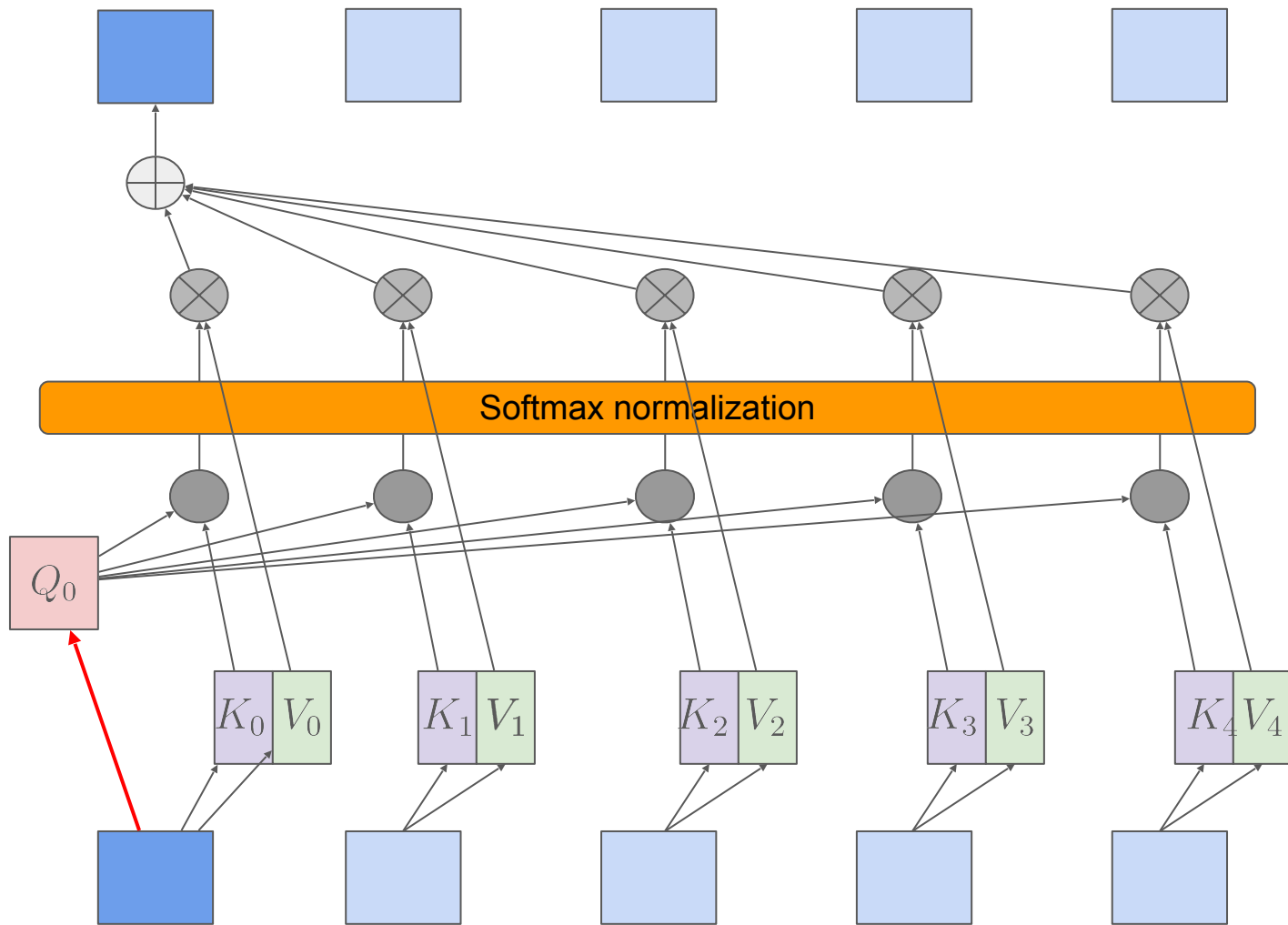


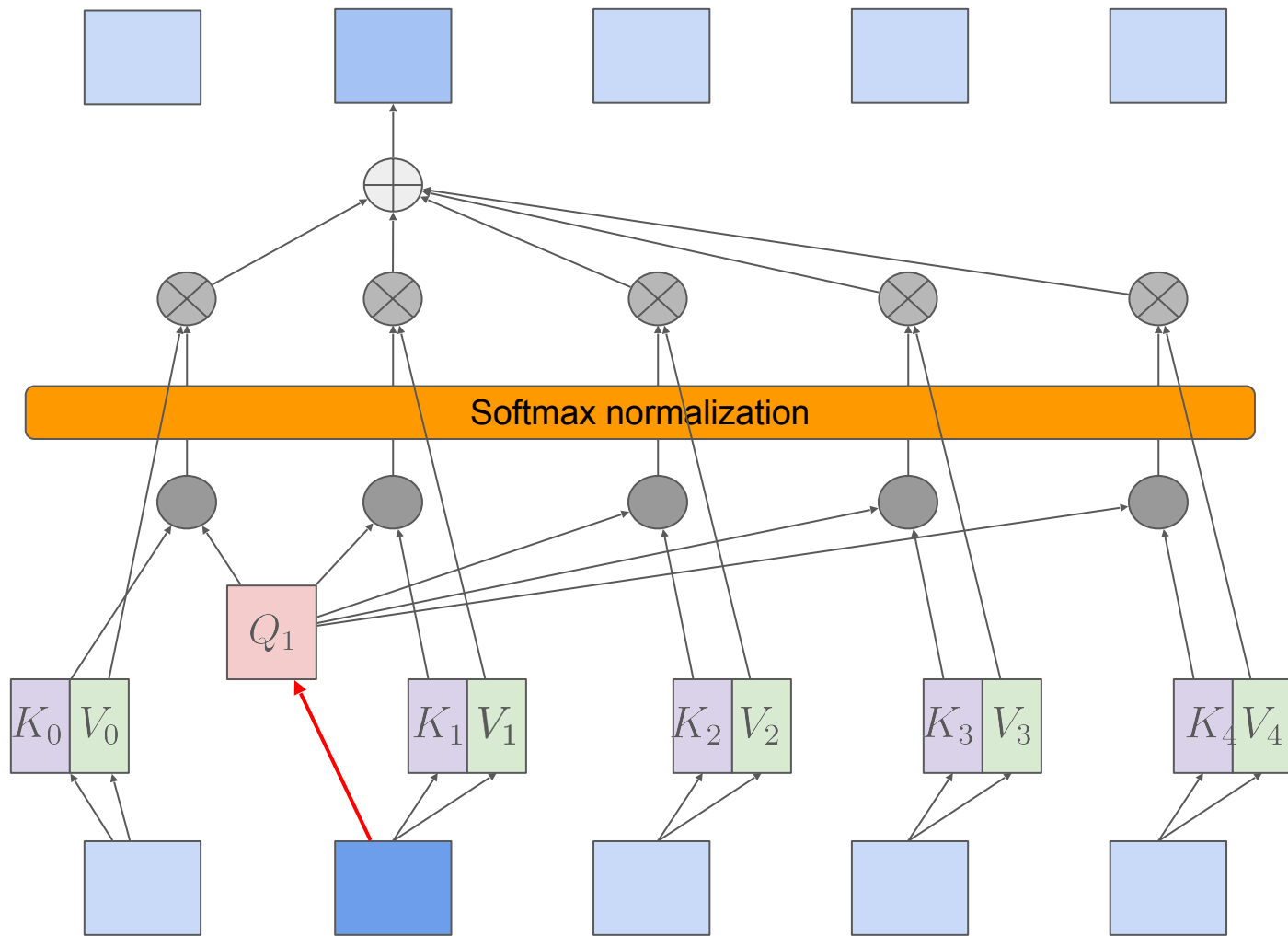


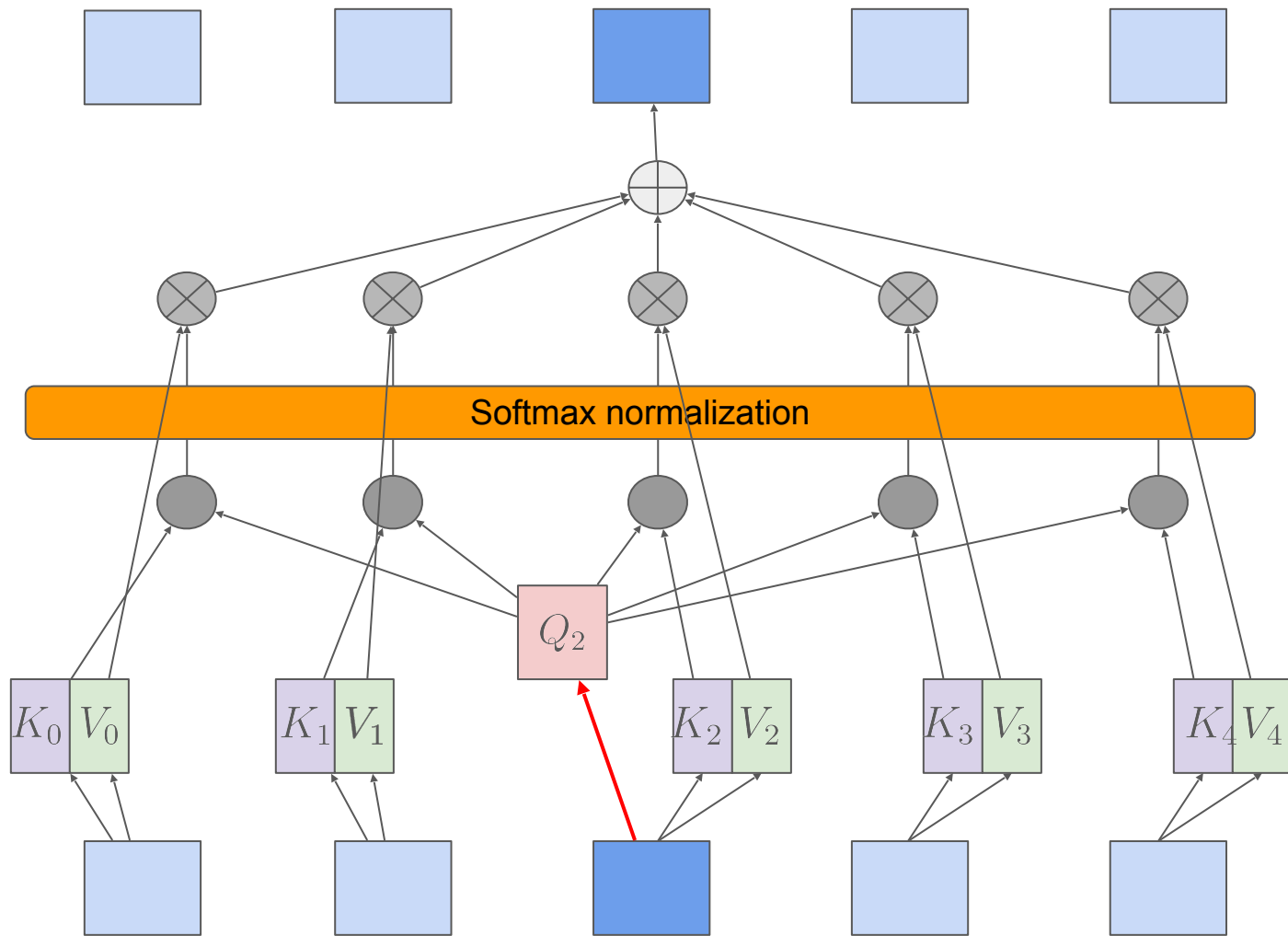


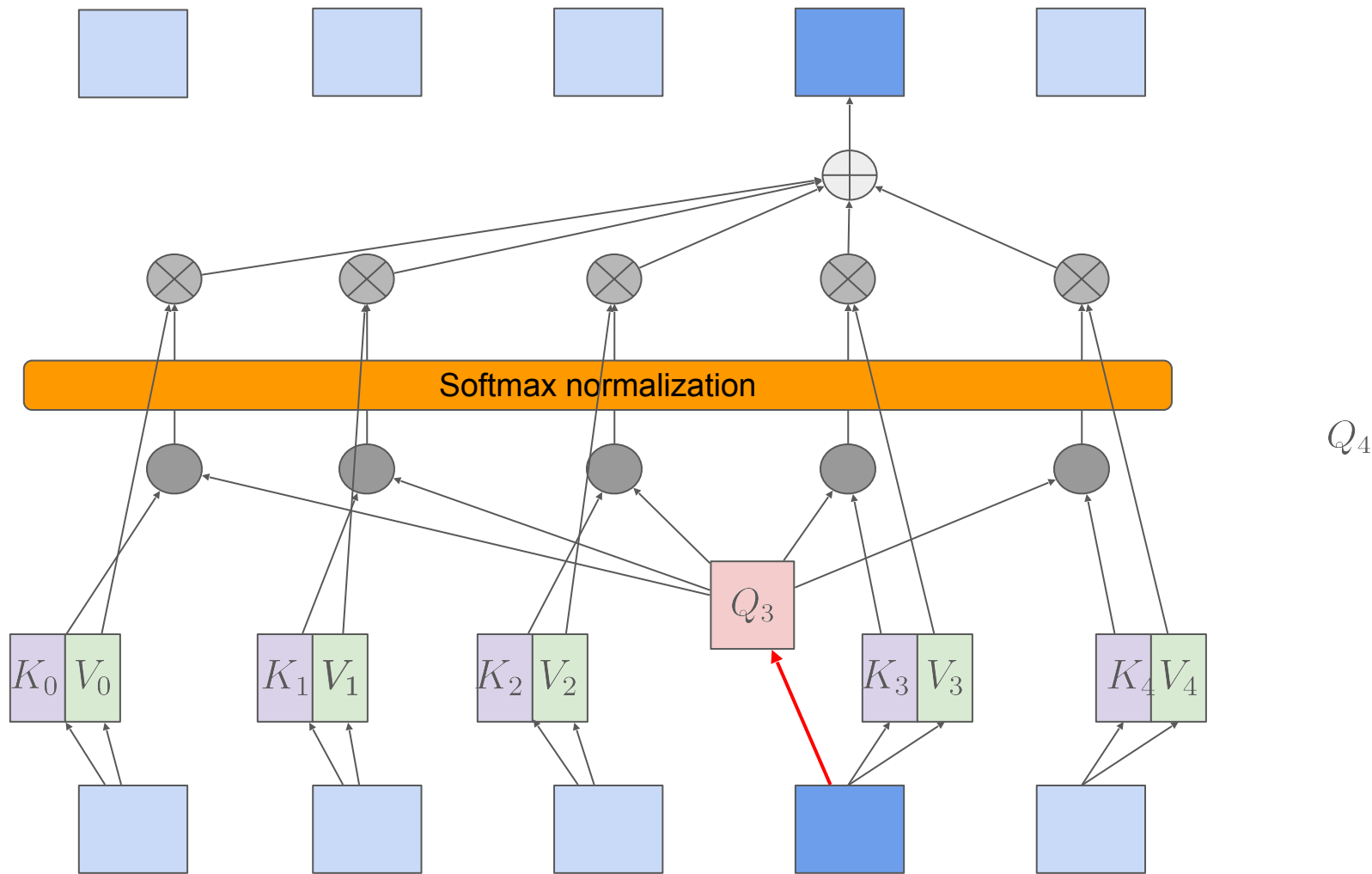


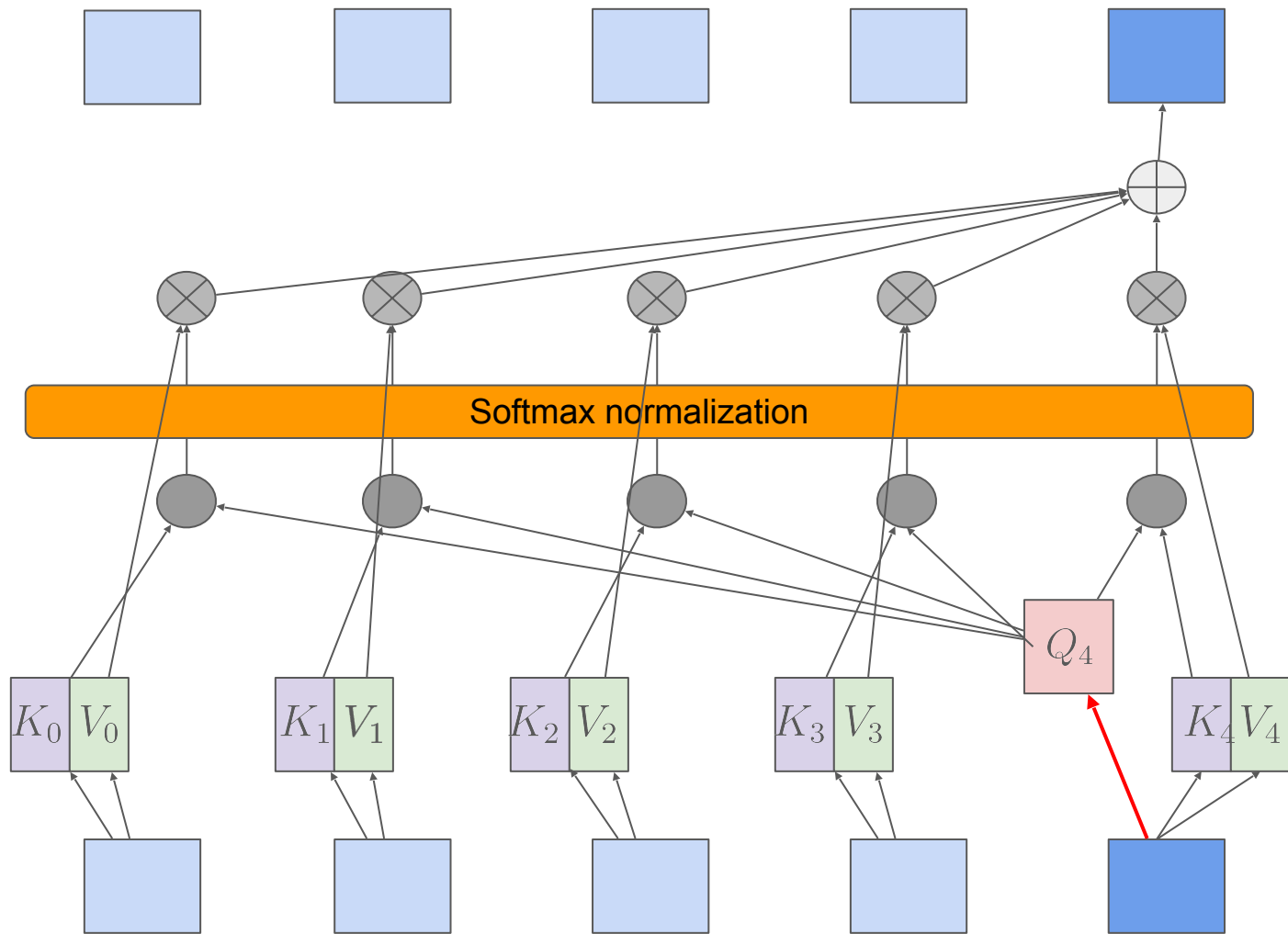












Where do Q, K, V come from?

I like white chocolate

Where do Q, K, V come from?

x_1

I like white chocolate

Where do Q, K, V come from?

x_1 x_2
I like white chocolate

Where do Q, K, V come from?

x_1	x_2	x_3	
I	like	white	chocolate

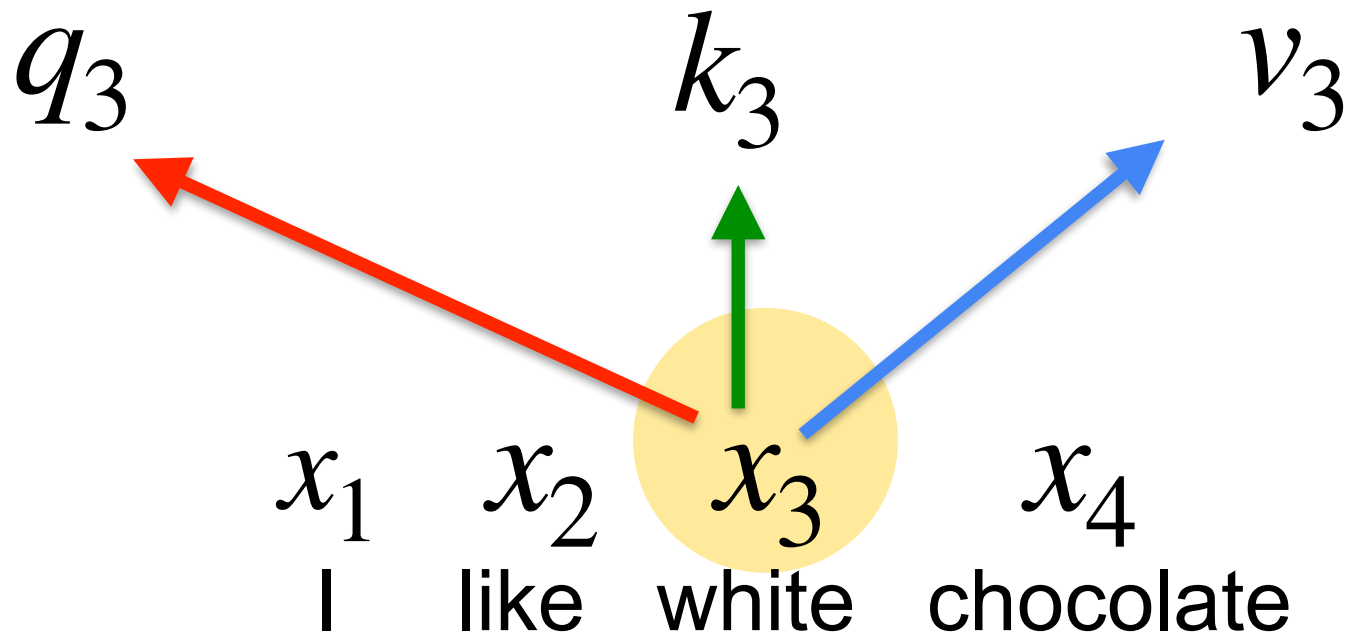
Where do Q, K, V come from?

x_1	x_2	x_3	x_4
I	like	white	chocolate

Where do Q, K, V come from?

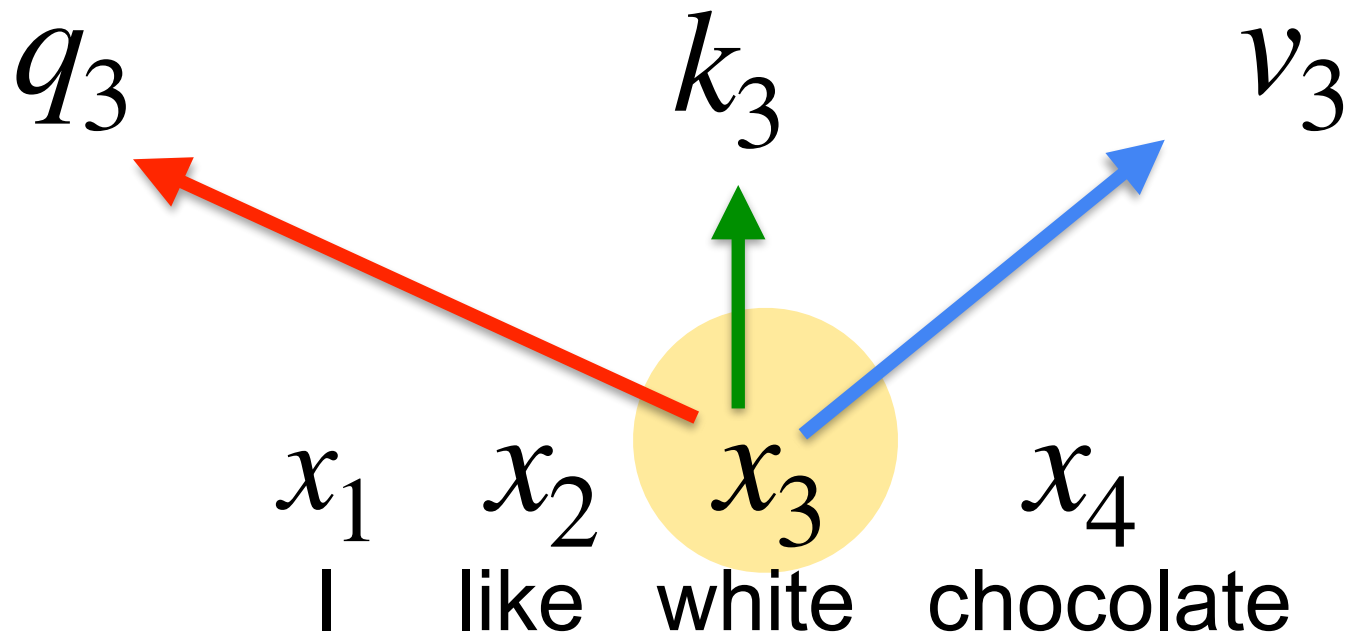
x_1	x_2	x_3	x_4
I	like	white	chocolate

Where do Q, K, V come from?



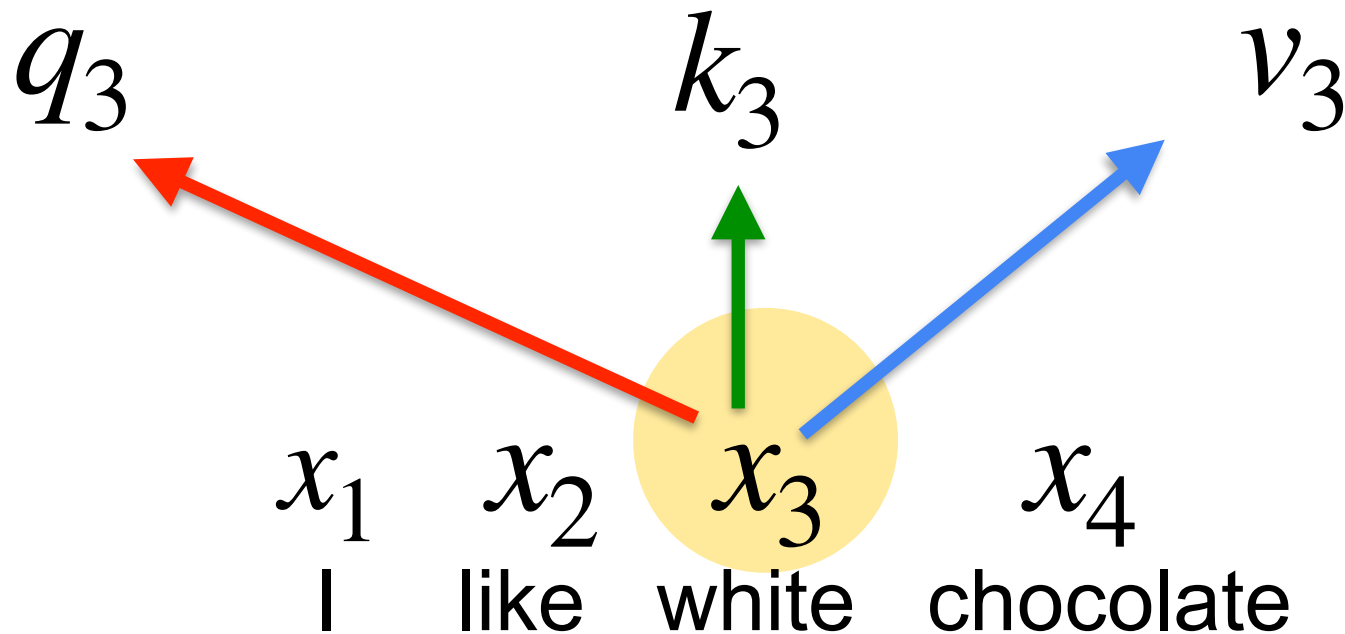
Where do Q, K, V come from?

$$q = W_Q x$$



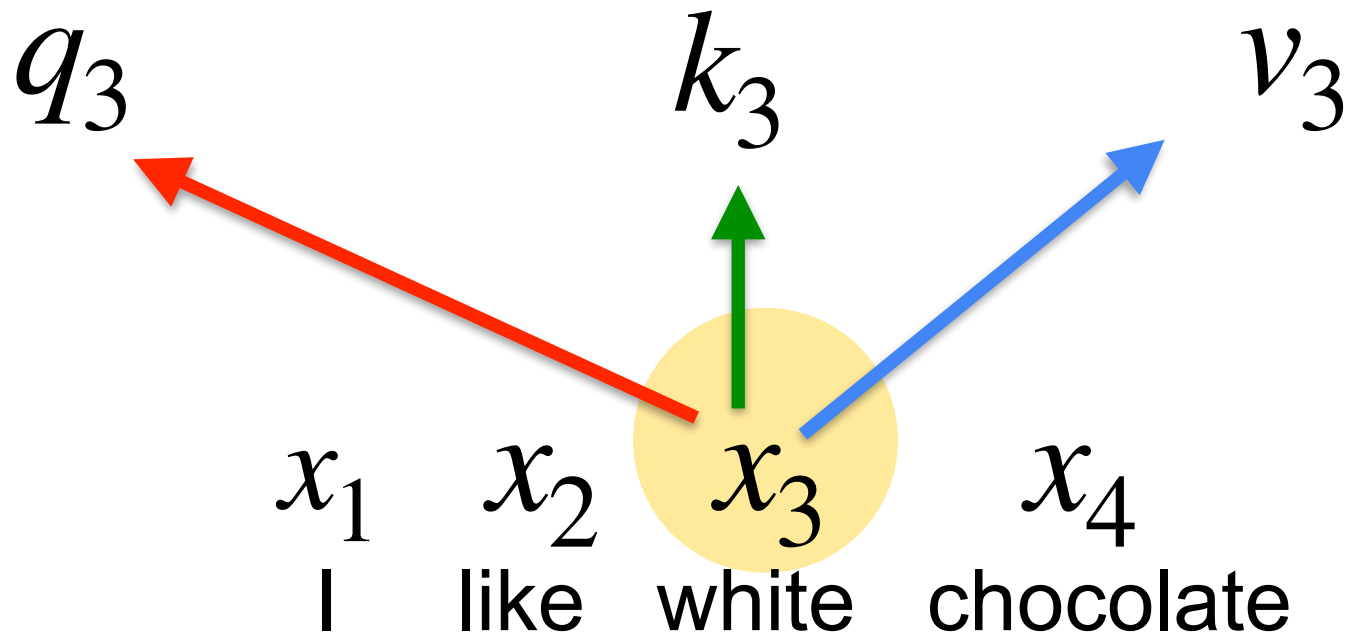
Where do Q, K, V come from?

$$q = W_Q x \quad k = W_K x$$



Where do Q, K, V come from?

$$q = W_Q x \quad k = W_K x \quad v = W_V x$$



Where do Q, K, V come from?

Learning

$$q = W_Q x$$

$$k = W_K x$$

$$v = W_V x$$

q_3

k_3

v_3

x_1

x_2

x_3

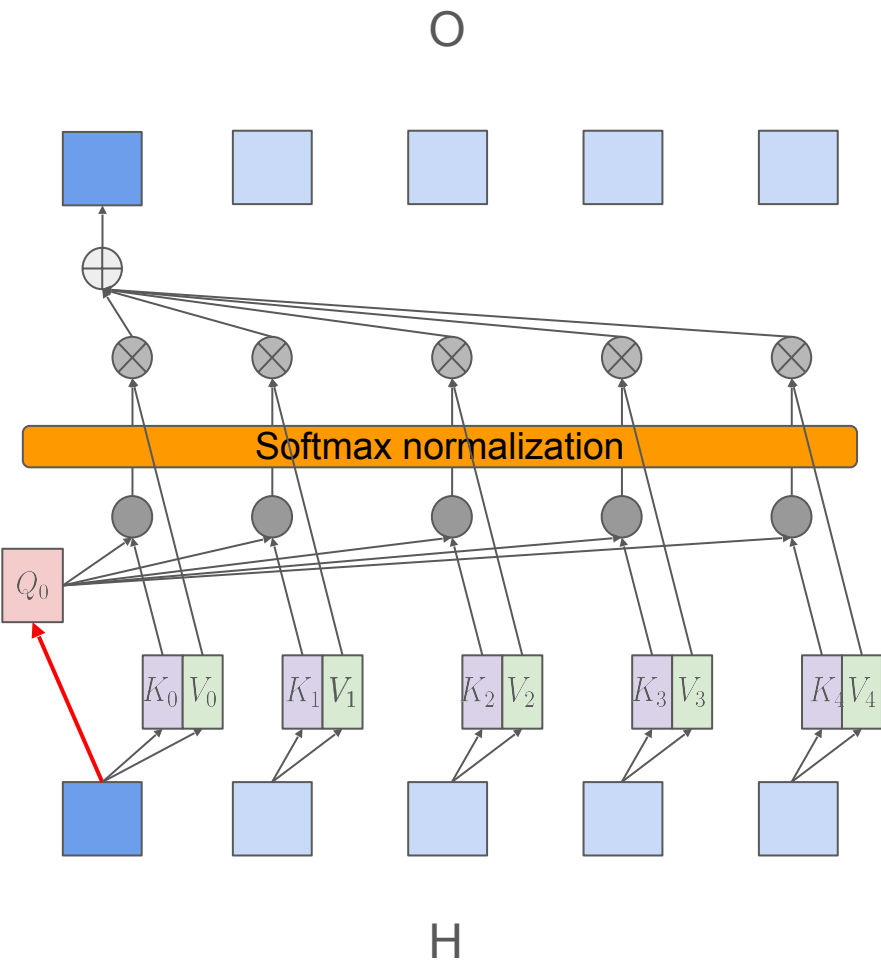
x_4

I

like

white

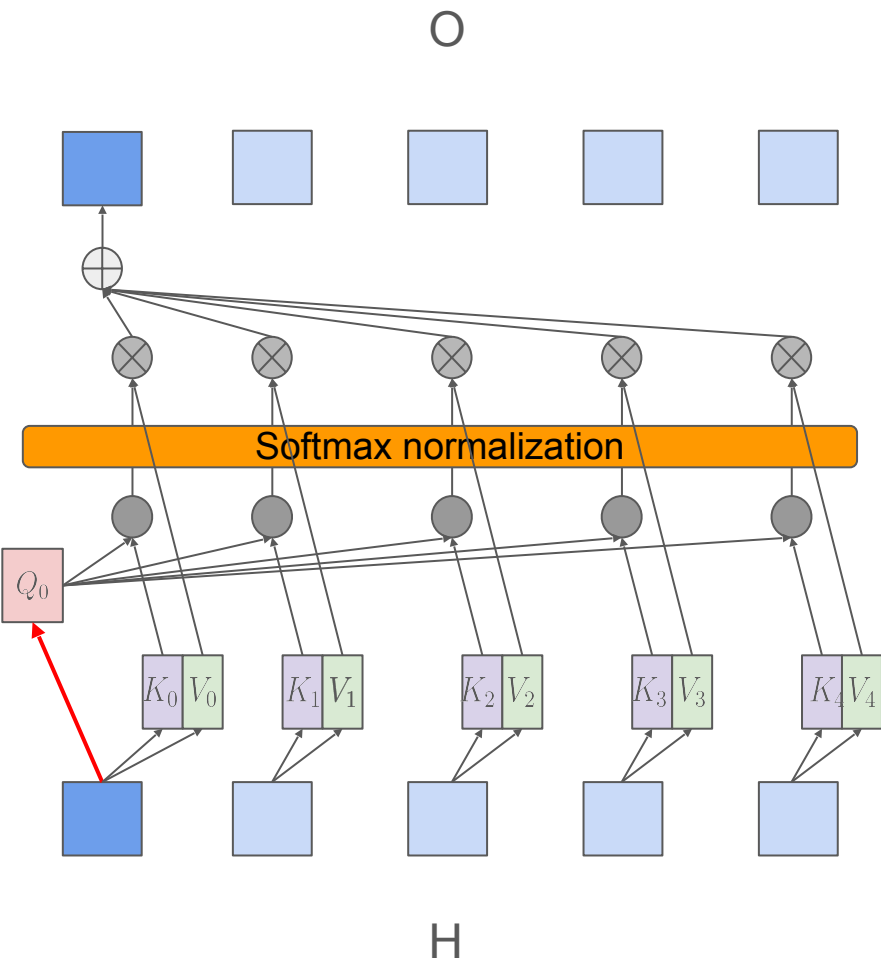
chocolate



$$Q = W_Q H \in \mathbb{R}^{n \times d}$$

$$K = W_K H \in \mathbb{R}^{n \times d}$$

$$V = W_V H \in \mathbb{R}^{n \times d}$$



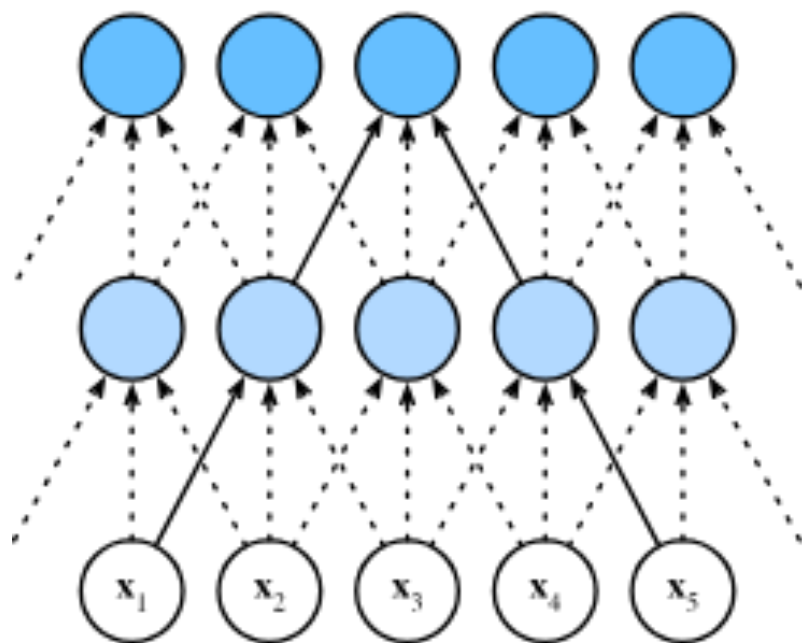
$$Q = W_Q H \in \mathbb{R}^{n \times d}$$

$$K = W_K H \in \mathbb{R}^{n \times d}$$

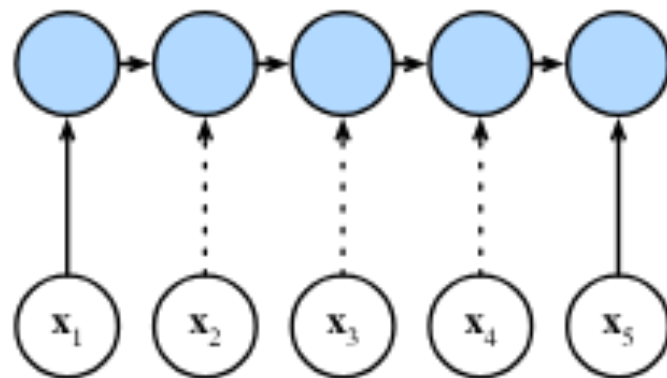
$$V = W_V H \in \mathbb{R}^{n \times d}$$

$$O = \text{softmax}\left(\frac{Q^T K}{\sqrt{d}}\right) V \in \mathbb{R}^{n \times d}$$

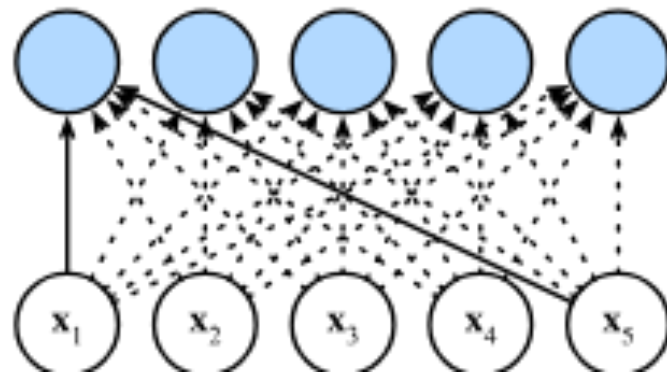
CNN

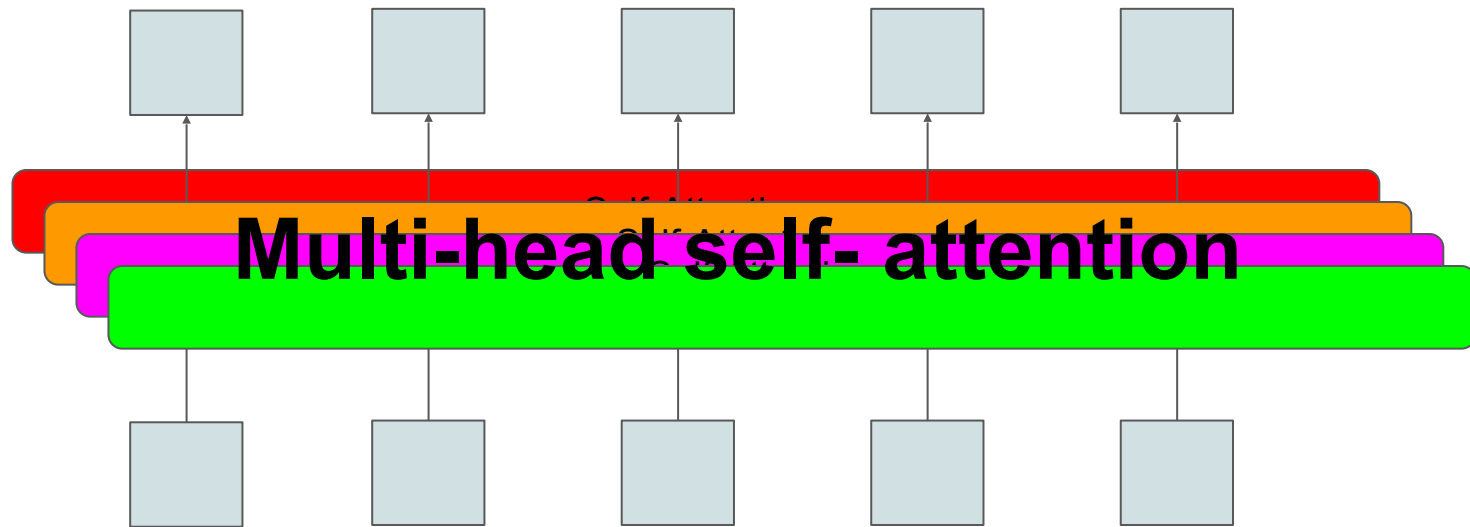


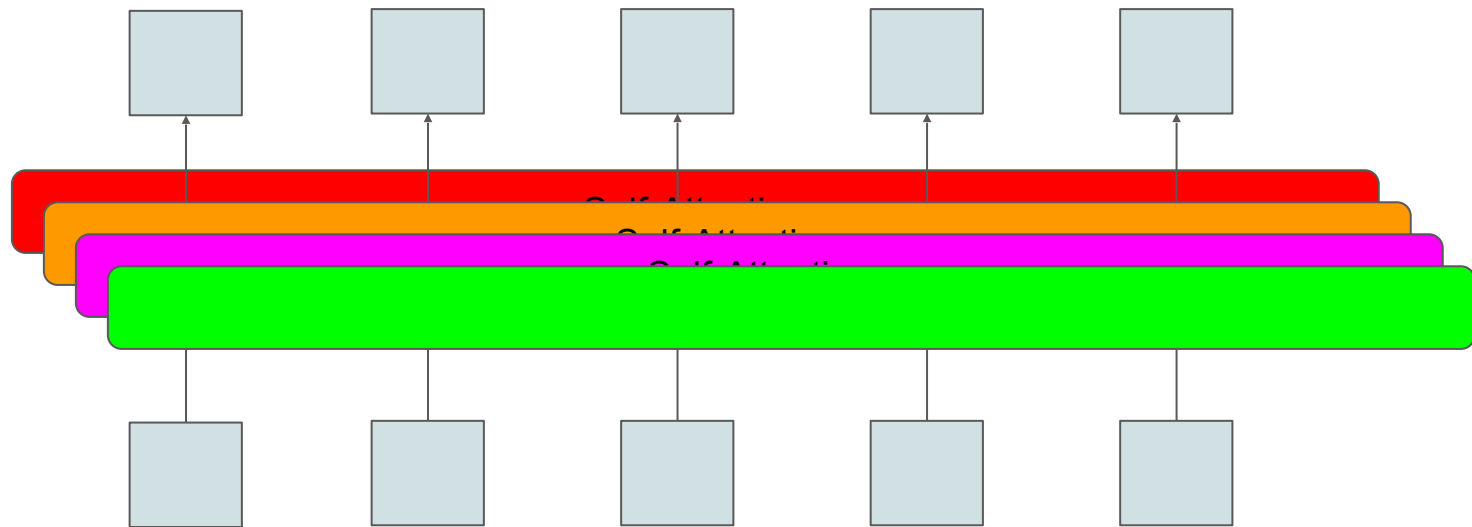
RNN



Self-attention







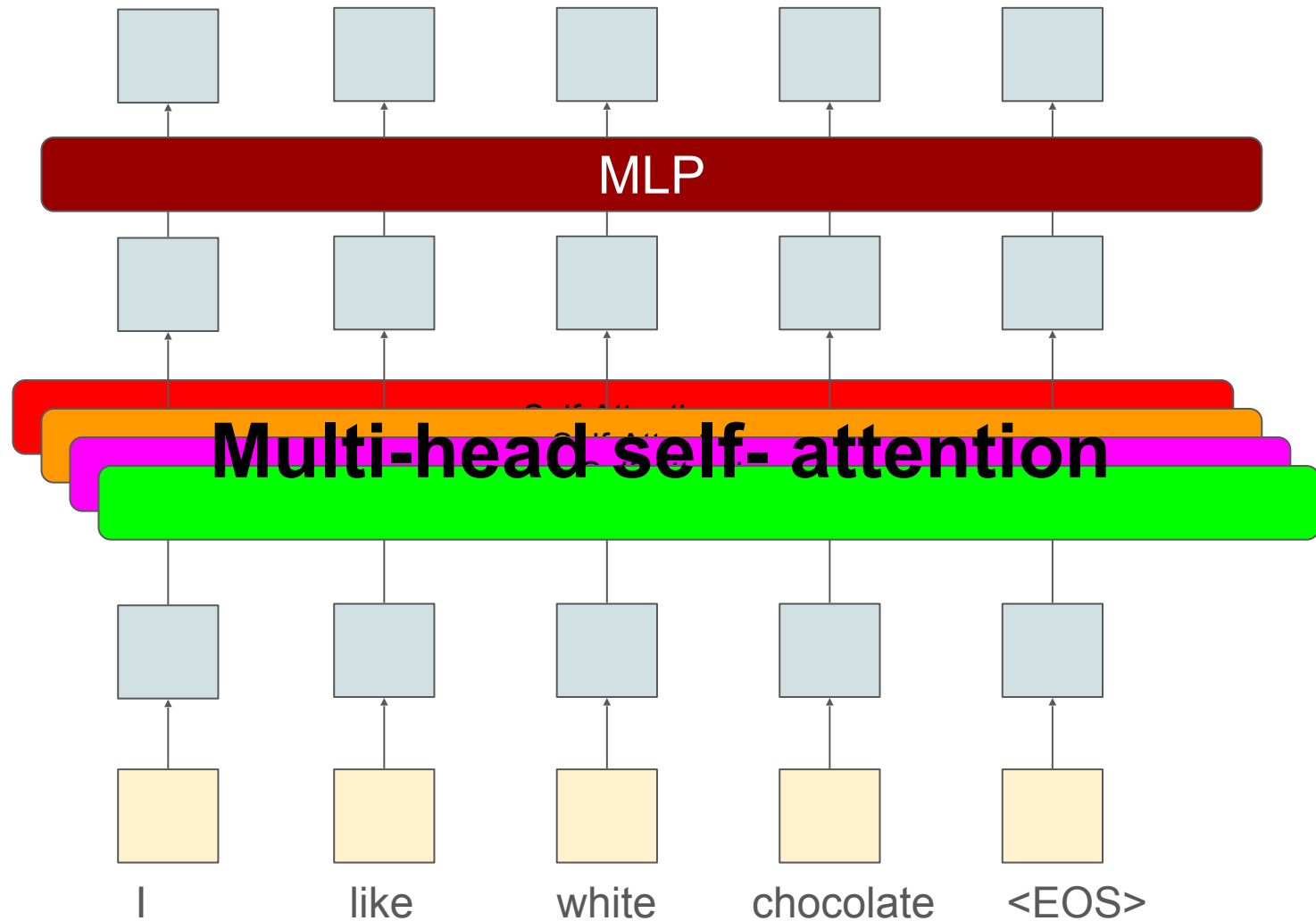
$$Q^i = W_Q^i H \in \mathbb{R}^{n \times d}$$

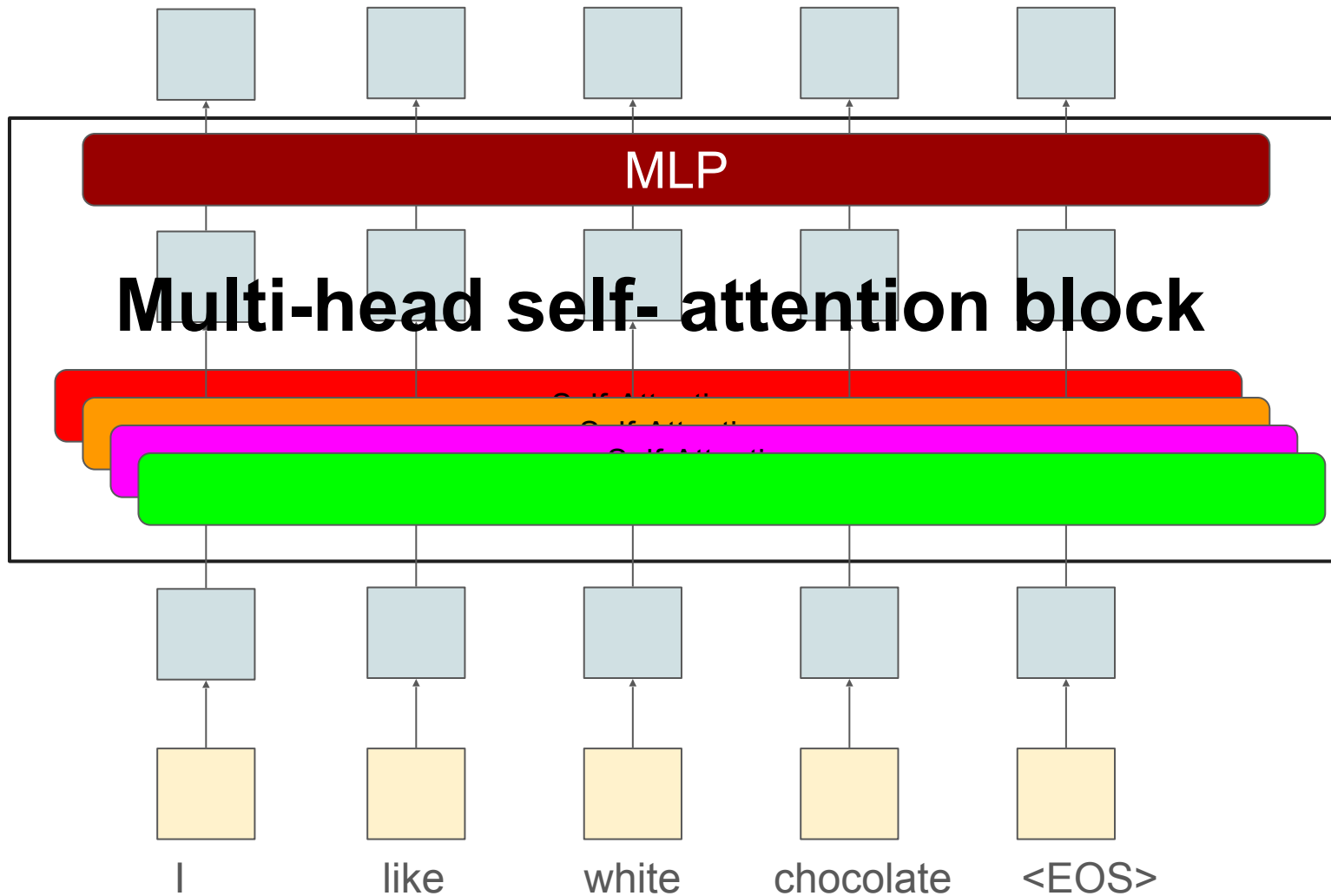
$$K^i = W_K^i H \in \mathbb{R}^{n \times d}$$

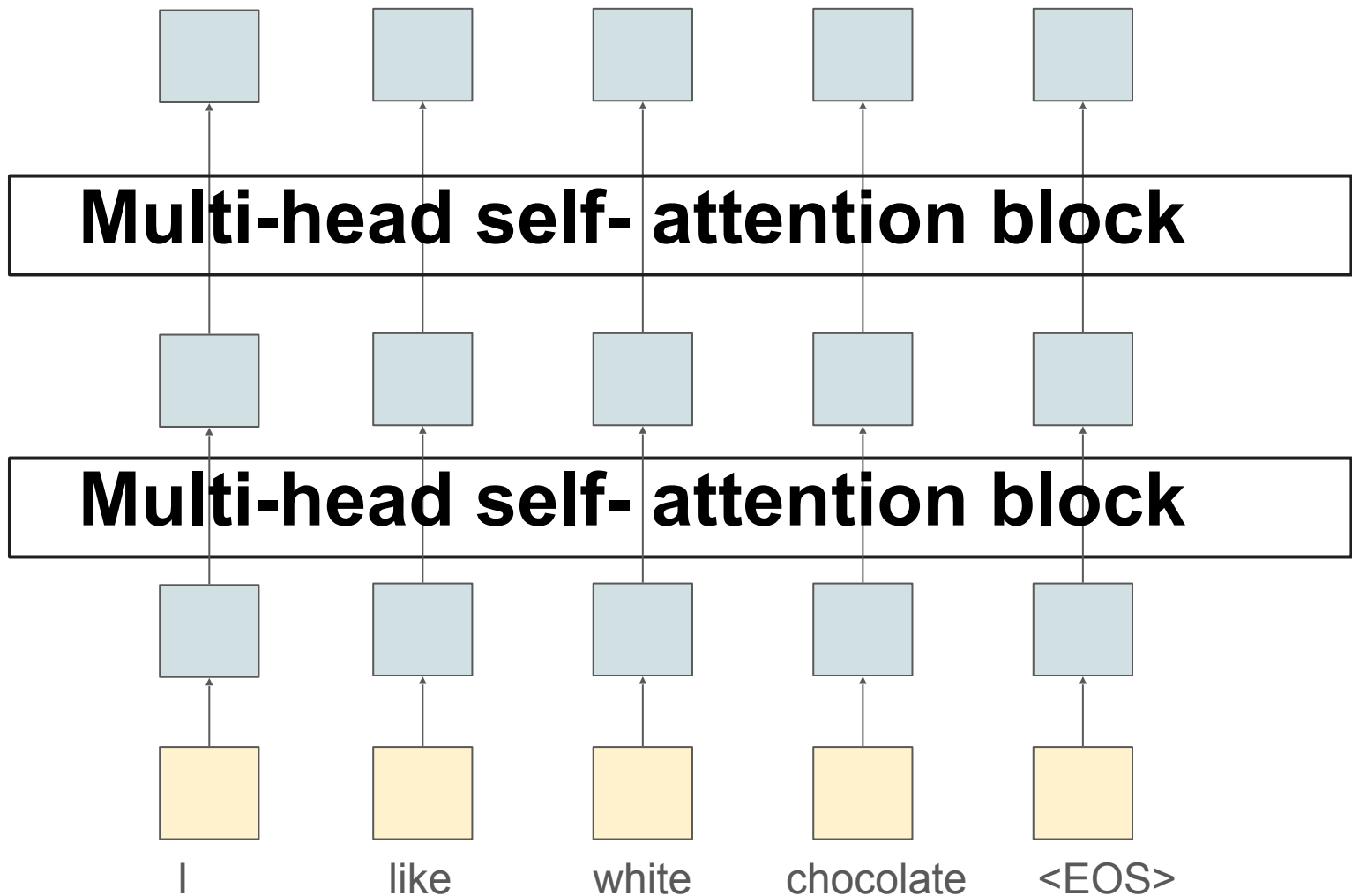
$$V^i = W_V^i H \in \mathbb{R}^{n \times d}$$

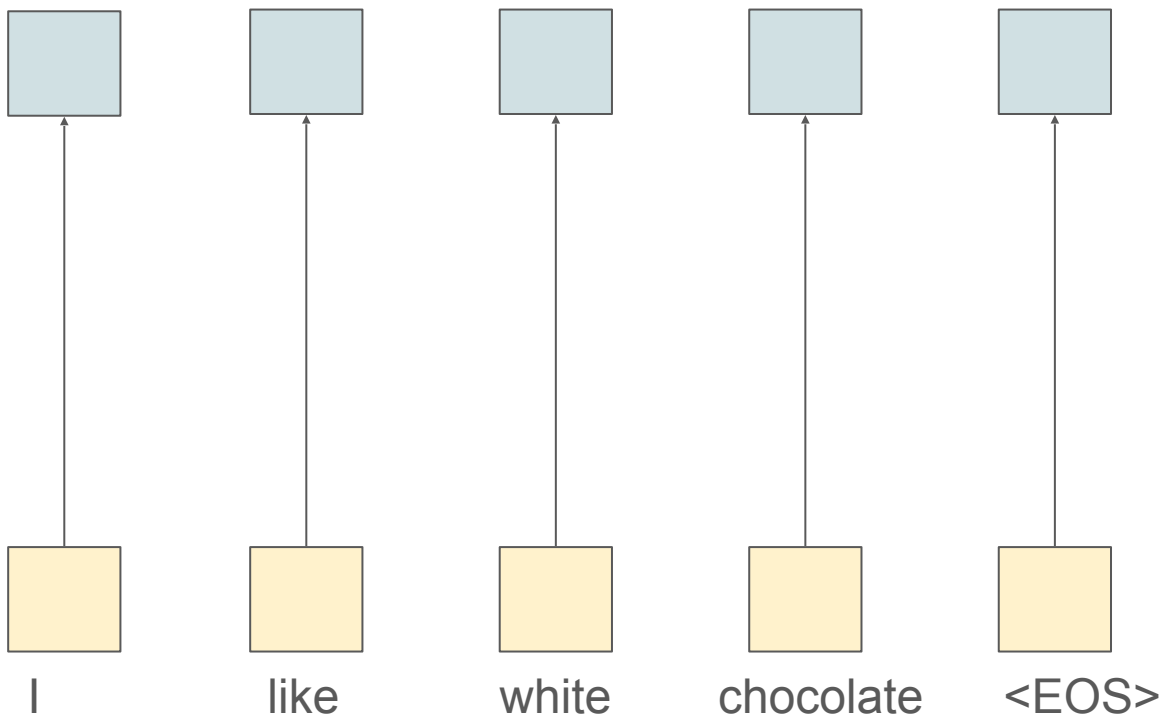
$$O^i = \text{softmax}\left(\frac{Q^T K}{\sqrt{d}}\right) V \in \mathbb{R}^{n \times d}$$

$$O = [O^0 O^1 \dots O^m]$$





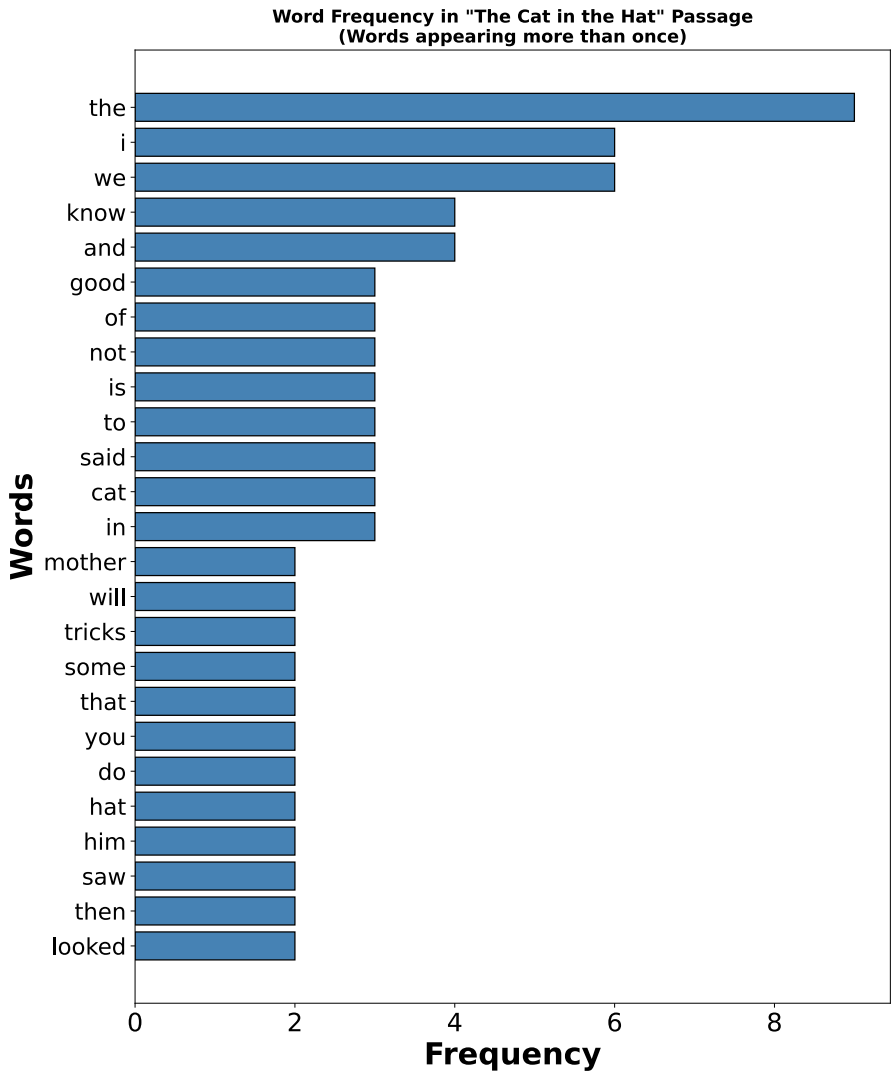




Tokenization

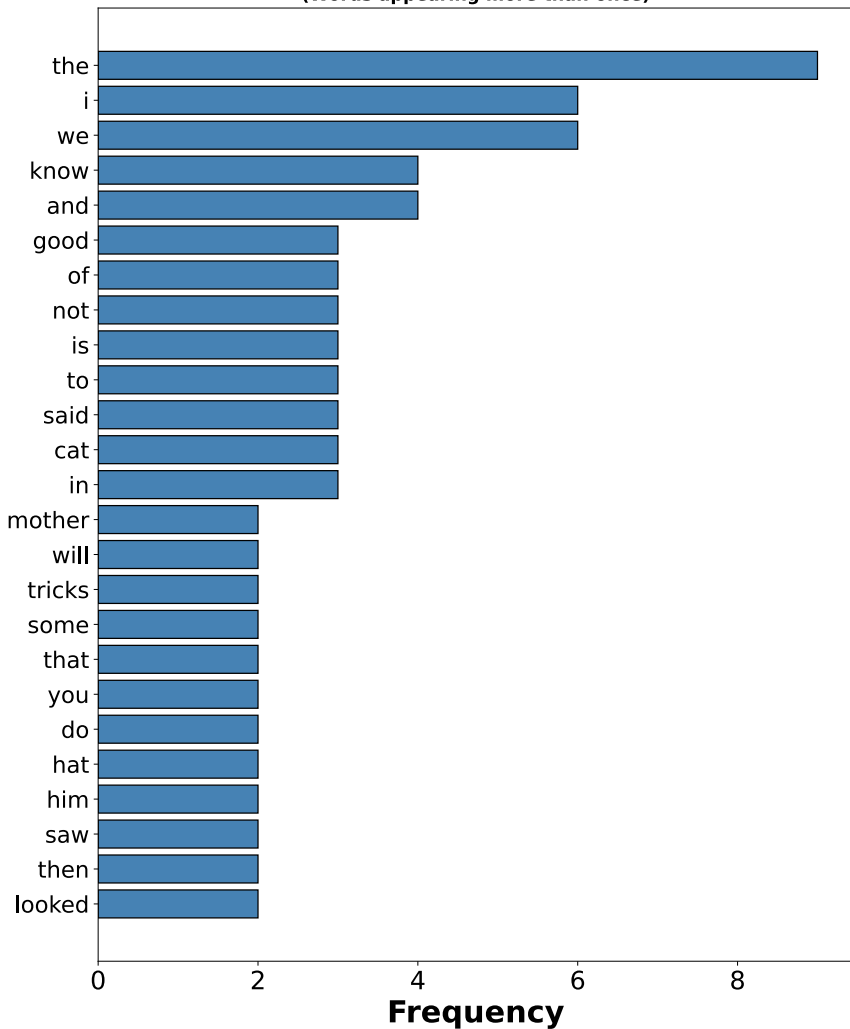
The Cat in the Hat, Dr Seuss

We looked! Then we saw him step in on the mat!
We looked! And we saw him! The Cat in the Hat!
And he said to us, "Why do you sit there like that?"
"I know it is wet And the sun is not sunny.
But we can have Lots of good fun that is funny!"
"I know some good games we could play," Said the cat.
"I know some new tricks," Said the Cat in the Hat.
"A lot of good tricks. I will show them to you.
Your Mother Will not mind at all if I do."
Then Sally and I Did not know what to say.
Our mother was out of the house For the day.



Word Frequency in "The Cat in the Hat" Passage
(Words appearing more than once)

Words



Representing each **word** with a vector

- Captures semantic connections
- Cannot deal with out-of-vocabulary words
- Cannot deal with punctuation, code, or other symbols
- Cannot deal with stemming

Stemming

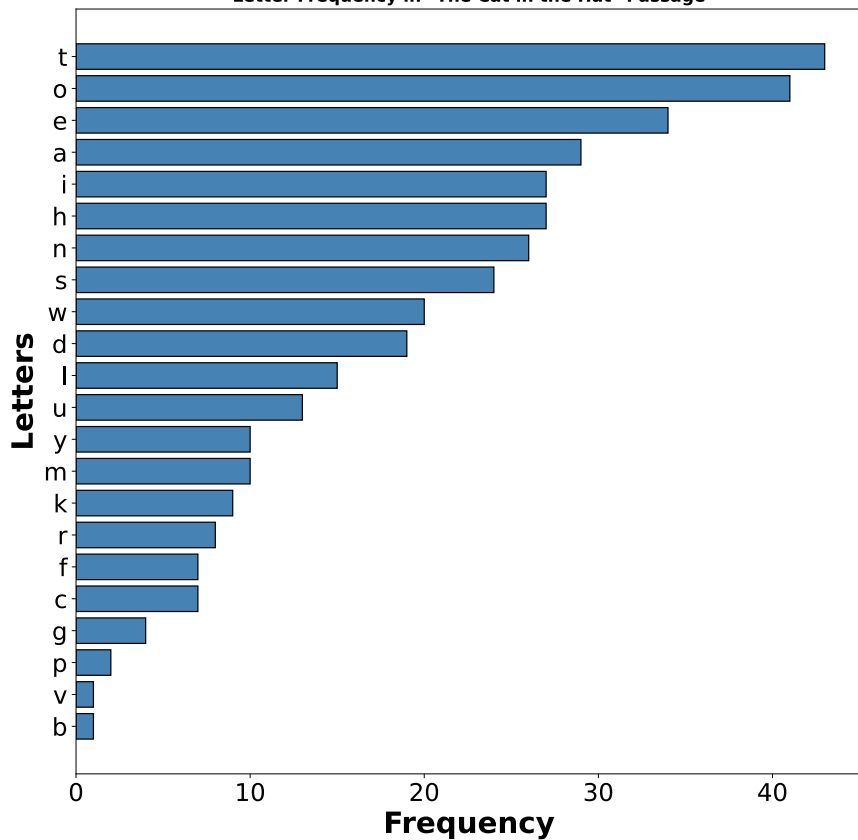
The teacher teaches students.
Teaching is rewarding when
teachers teach effectively. She
taught mathematics and enjoys
teaching it. Her teachings
influence how students learn.

Stemming

The **teacher teaches** students.

Teaching is rewarding when
teachers teach effectively. She
taught mathematics and enjoys
teaching it. Her **teachings**
influence how students learn.

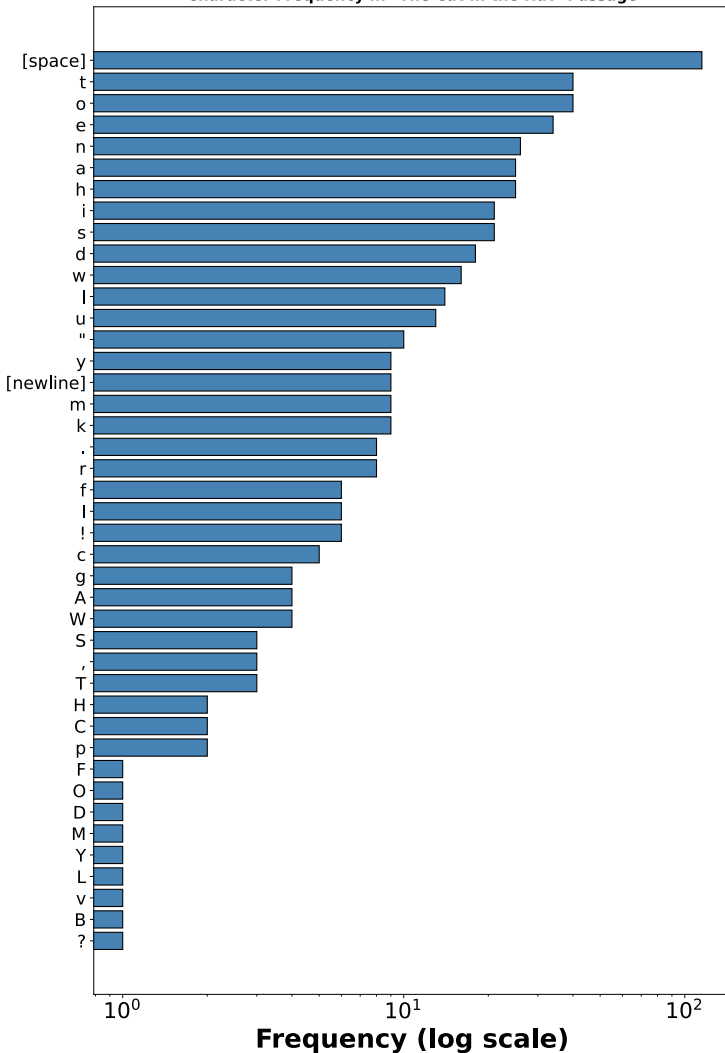
Letter Frequency in "The Cat in the Hat" Passage



Representing each **letter** with a vector

- Does not capture semantic connections
- Out-of-vocabulary letters are very rare
- Cannot deal with punctuation, code, or other symbols
- Cannot deal with stemming

Character Frequency in "The Cat in the Hat" Passage



Representing each **character** with a vector

- Does not captures semantic connections
- Out-of-vocabulary characters are very rare
- Can deal with punctuation, code, or other symbols
- Cannot deal with stemming

Stemming

The **teacher teaches** students.

Teaching is rewarding when
teachers teach effectively. She
taught mathematics and enjoys
teaching it. Her **teachings**
influence how students learn.

Stemming

The **teacher teaches** students.

Teaching is rewarding when
teachers teach effectively. She
taught mathematics and enjoys
teaching it. Her **teachings**
influence how students learn.

teach

Stemming

The **teacher teaches** students.
Teaching is rewarding when
teachers teach effectively. She
taught mathematics and enjoys
teaching it. Her **teachings**
influence how students learn.

teach
teacher

Stemming

The **teacher teaches** students.
Teaching is rewarding when
teachers teach effectively. She
taught mathematics and enjoys
teaching it. Her **teachings**
influence how students learn.

teach
teacher
teaches

Stemming

The **teacher teaches** students.
Teaching is rewarding when
teachers teach effectively. She
taught mathematics and enjoys
teaching it. Her **teachings**
influence how students learn.

teach
teacher
teaches
taught

Stemming

The **teacher teaches** students.
Teaching is rewarding when
teachers teach effectively. She
taught mathematics and enjoys
teaching it. Her **teachings**
influence how students learn.

teach
teacher
teaches
taught
teachings

Stemming

The **teacher teaches** students.
Teaching is rewarding when
teachers teach effectively. She
taught mathematics and enjoys
teaching it. Her **teachings**
influence how students learn.

teach
teacher
teaches
taught
teachings
teachers

Stemming

The **teacher teaches** students.
Teaching is rewarding when
teachers teach effectively. She
taught mathematics and enjoys
teaching it. Her **teachings**
influence how students learn.

teach
teacher
teaches
taught
teachings
teachers

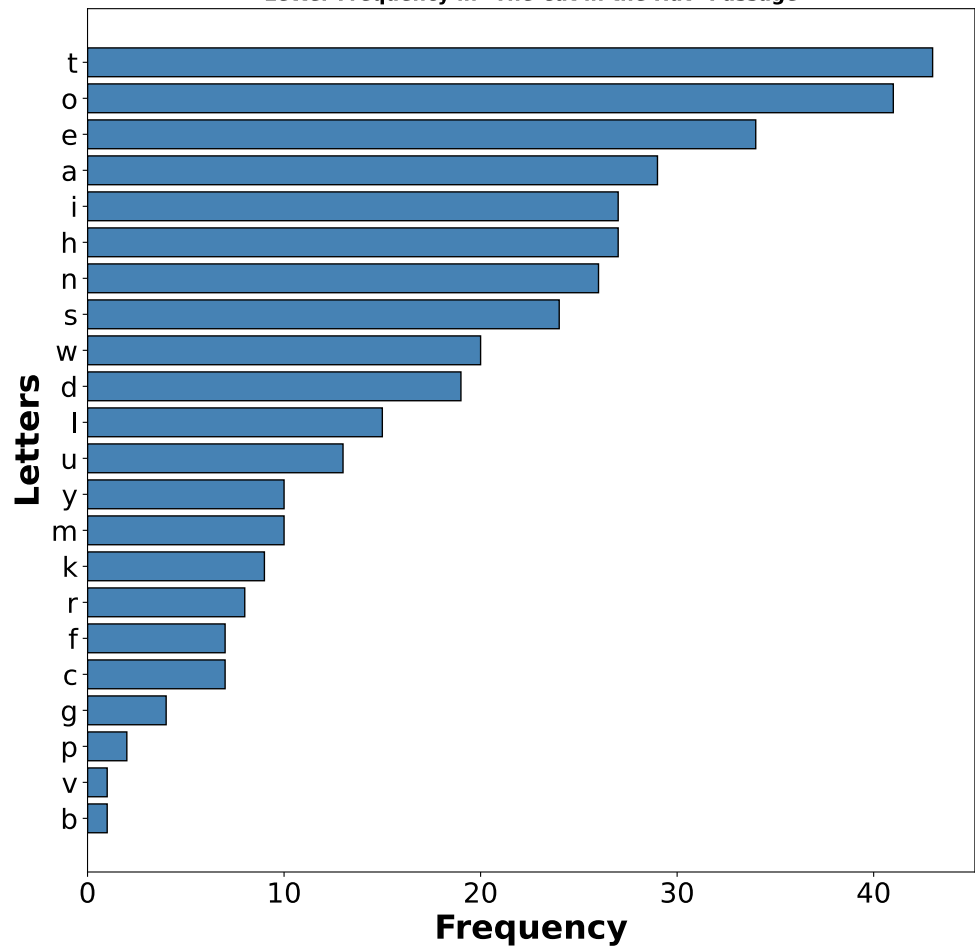


“Teach” is frequent

Frequent pairings

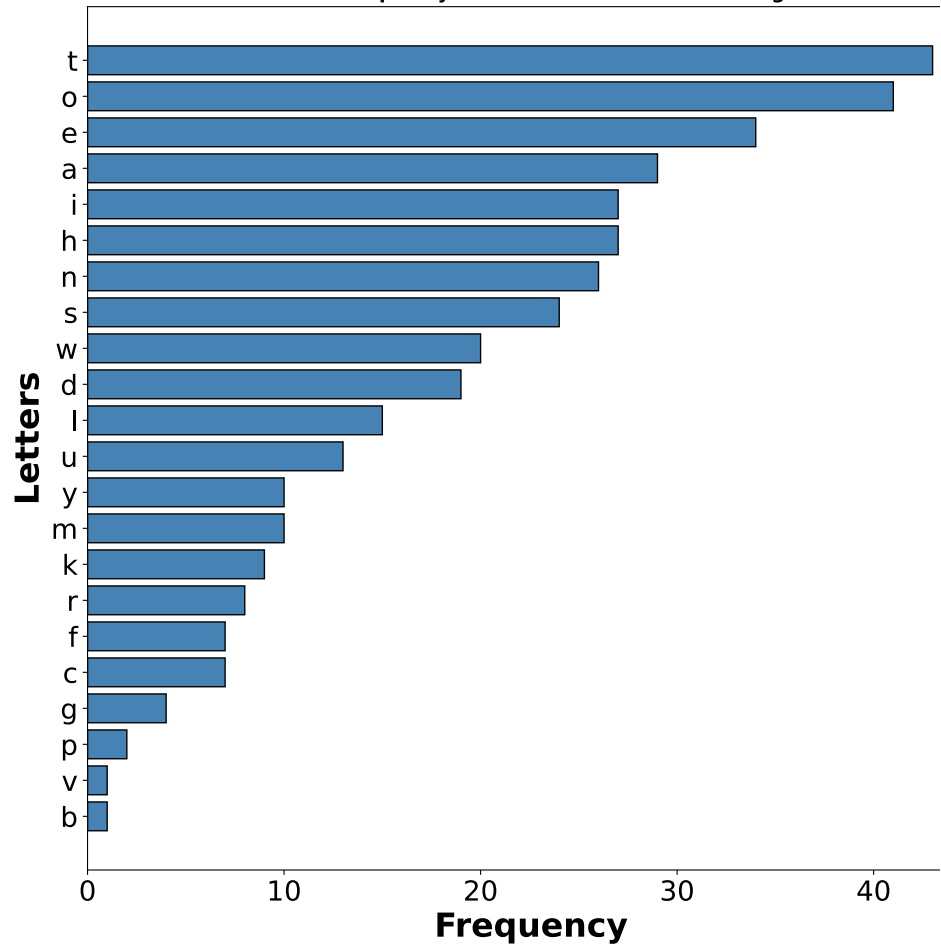
Frequent pairings

Letter Frequency in "The Cat in the Hat" Passage

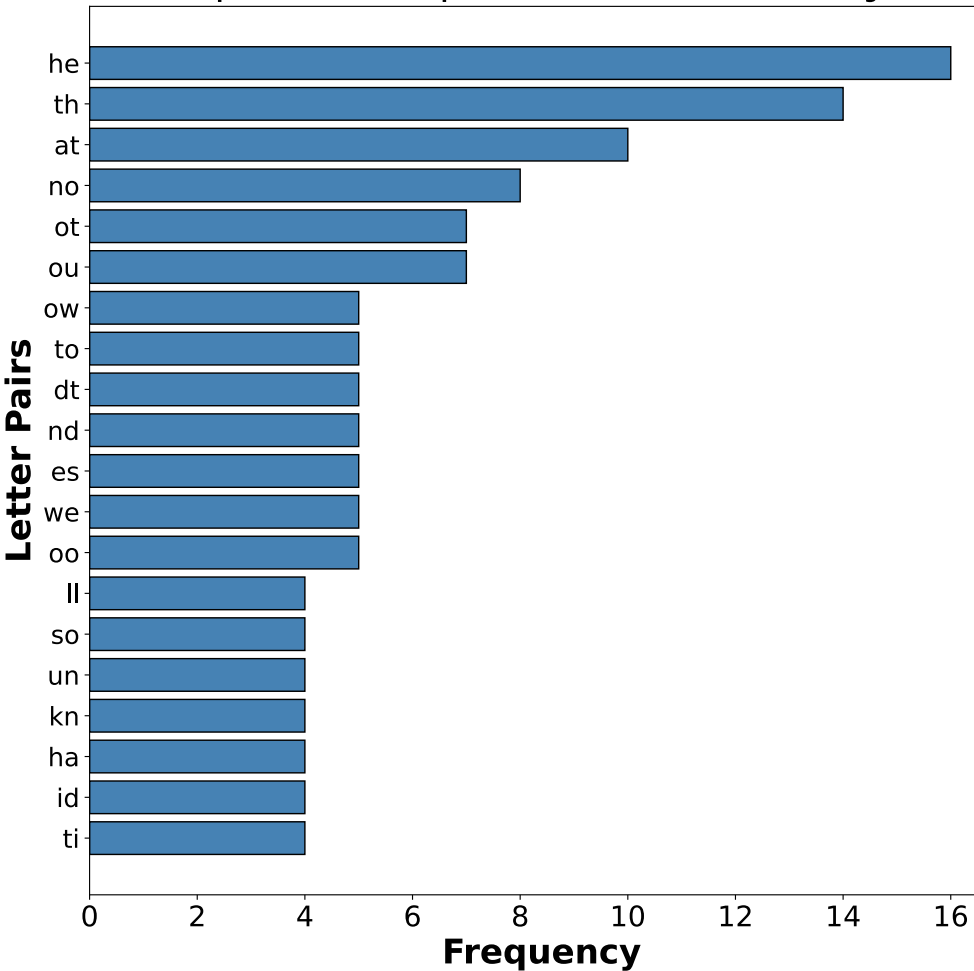


Frequent pairings

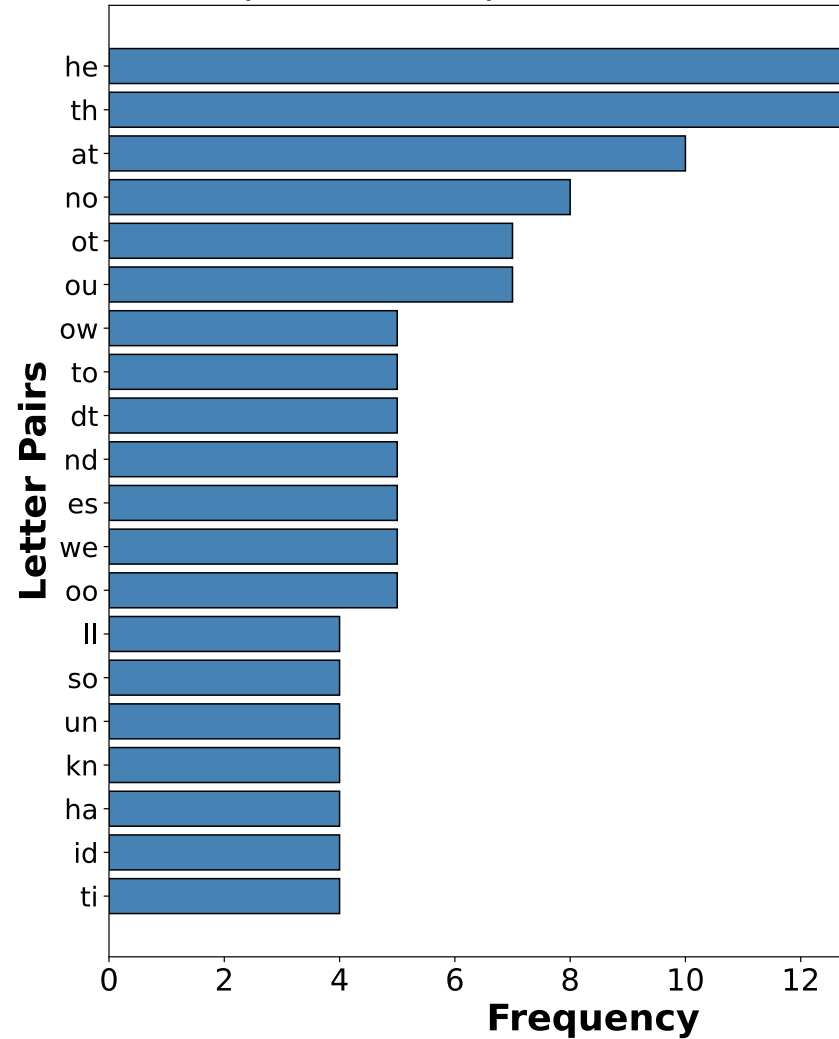
Letter Frequency in "The Cat in the Hat" Passage



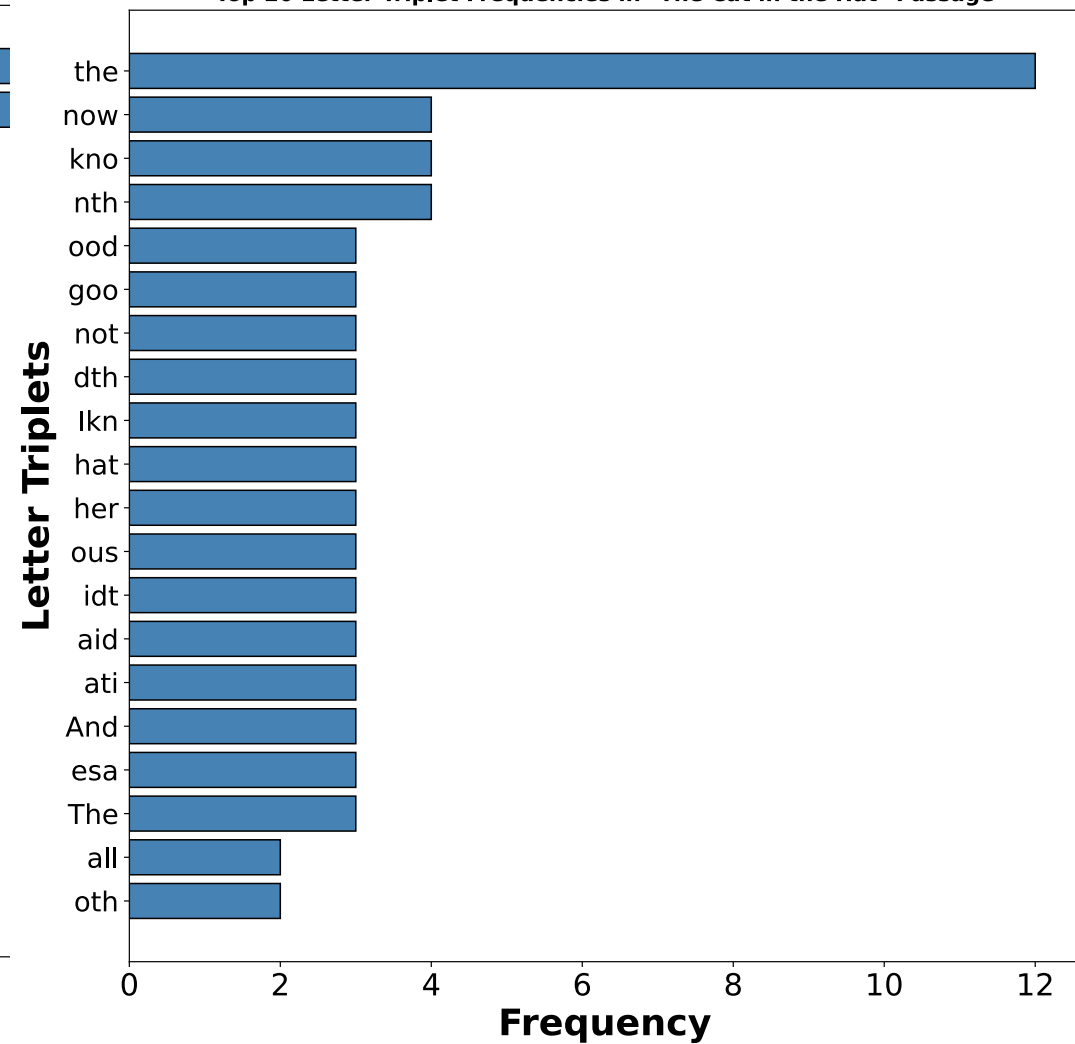
Top 20 Letter Pair Frequencies in "The Cat in the Hat" Passage

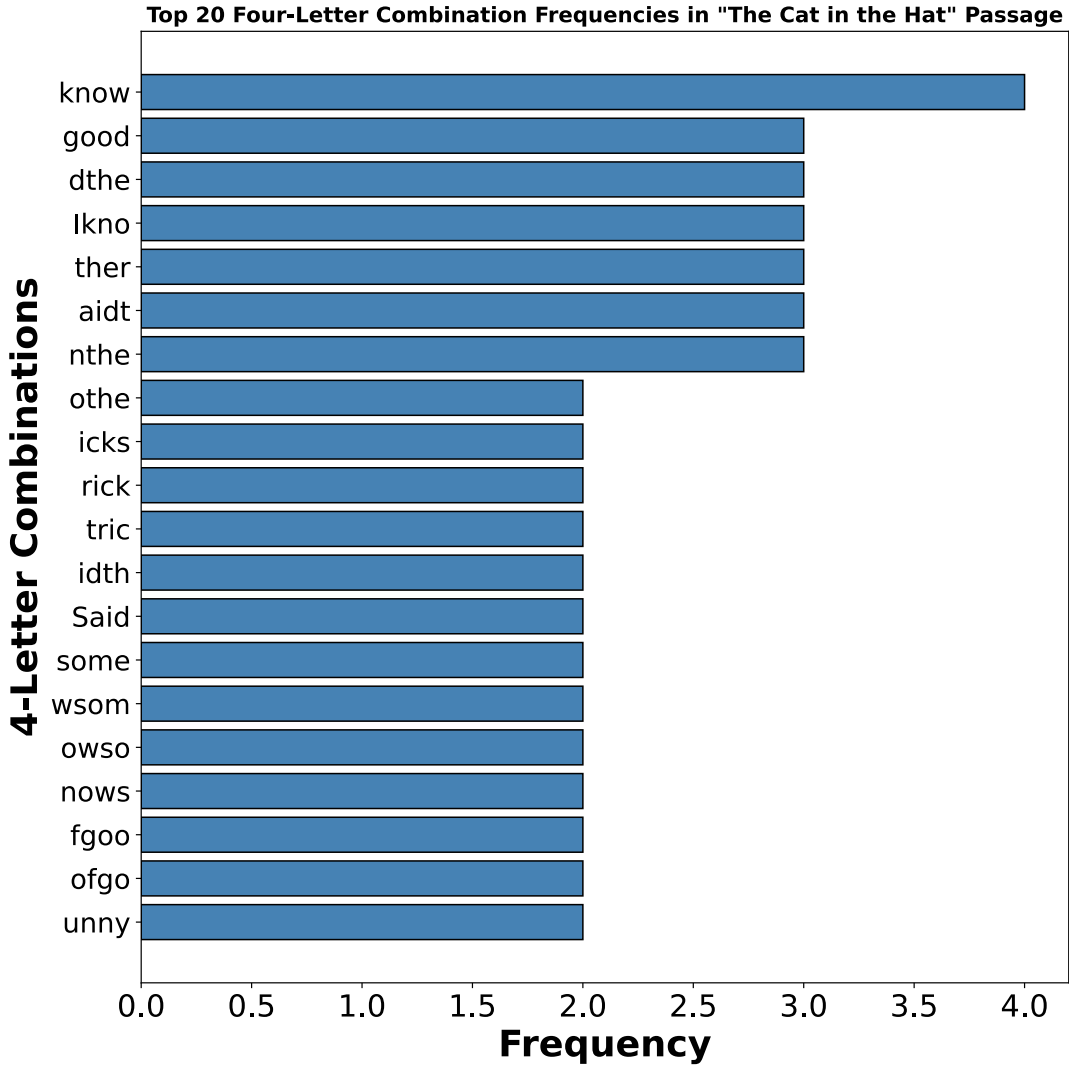
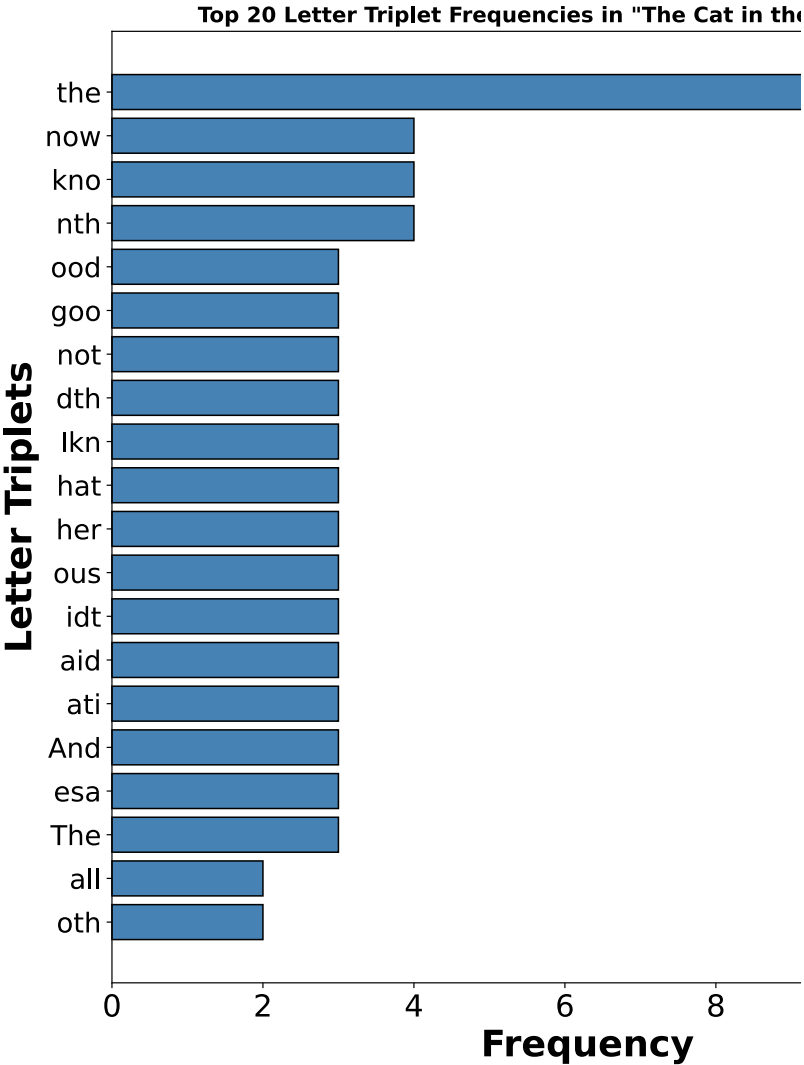


Top 20 Letter Pair Frequencies in "The Cat in the Hat"



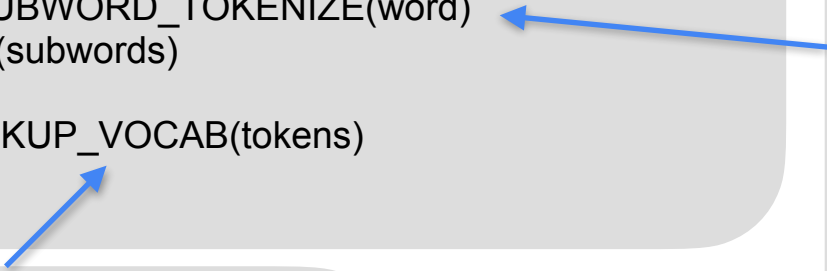
Top 20 Letter Triplet Frequencies in "The Cat in the Hat" Passage





Tokenizing text

```
function TOKENIZE(text):  
    ``Take text and transform it into a list of tokens``  
    text = NORMALIZE(text)          # transform text into unicode  
    words = PRETOKENIZE(text)       # split words at blanks,  
                                    # punctuation -> words  
  
    tokens = []  
  
    for word in words:  
        subwords = SUBWORD_TOKENIZE(word)  
        tokens.extend(subwords)
```



```
function LOOKUP_VOCAB(tokens):  
    ``Replace tokens with vectors``  
    ids = []  
    for token in tokens:  
        ids.append(vocab[token])  
    return ids
```

```
function SUBWORD_TOKENIZE(word):  
    ``Transform words into tokens``  
    tokens = []  
    i = 0  
  
    while i < length(word):  
        matched = false  
  
        for j from length(word) down to i+1:  
            substring = word[i:j]  
  
            if substring in vocab:  
                tokens.append(substring)  
                i = j  
                matched = true  
                break  
  
        if not matched:  
            tokens.append(UNK)  
            break  
  
    return tokens
```

```
function TRAIN_BPE(corpus, vocab_size):
```

```
# 1) Preprocess: make a word-frequency table
word_freq = COUNT_WORDS(corpus) # map: word -> count
```

```
# 2) Initialize each word as a sequence of symbols (characters),
#    often with an end-of-word marker "</w>"
symbols = {}
for (word, c) in word_freq:
    symbols[word] = list(chars(word)) + ["</w>"]

merges = [] # list of learned merges (a, b) -> ab
```

```
# 3) Iteratively add merges until we hit vocab_size
while CURRENT_VOCAB_SIZE(symbols) < vocab_size:
```

```
    # map: (sym_i, sym_{i+1}) -> count
    pair_freq = MAP_DEFAULT_0()
```

```
    # 3a) Count adjacent pairs weighted by word counts
    for (word, c) in word_freq:
        seq = symbols[word]
        for i in 0 .. len(seq)-2:
            pair = (seq[i], seq[i+1])
            pair_freq[pair] += c
```

```
    # 3b) Pick the most frequent pair
    best_pair = ARGMAX(pair_freq)
    if pair_freq[best_pair] == 0:
        break
```

```
    merges.append(best_pair)
```

```
    # 3c) Replace all occurrences of best_pair with
    #       merged symbol in all words
    for (word, c) in word_freq:
        symbols[word] = MERGE_PAIR(symbols[word], best_pair)
```

```
# 4) Build final vocab: all symbols that appear
#    in any word sequence
vocab = EXTRACT_SYMBOLS(symbols)
return vocab, merges
```

```
function MERGE_PAIR(seq, pair=(A,B)):
```

```
    out = []
    i = 0
    while i < len(seq):
        if i < len(seq)-1 and seq[i] == A and seq[i+1] == B:
            out.append(CONCAT(A,B)) # merged token "AB"
            i += 2
        else:
            out.append(seq[i])
            i += 1
    return out
```

Byte pair encoding (BPE) training (simplified)

- Start with tokens=characters
- Find the most frequently occurring pair of consecutive tokens
- Merge them into a new token, add it to the list of tokens
- Apply tokenization to the passage. (When multiple tokenizations are possible, apply rules in the order they were learned.)
- Stop when the token vocabulary reaches the desired size

Byte pair encoding inference (simplified)

- Start with characters
- Apply tokenization to the passage. When multiple tokenizations are possible, apply rules in the order they were learned

Byte pair encoding

Vocabulary size = 100

[We', 'loo', 'ke', 'd', '!', 'Then', 'we', 'saw', 'hi', 'm', 's', 't', 'e', 'p', 'in', 'o', 'n', 'the', 'm', 'at', '!', 'We', 'loo', 'ke', 'd', '!', 'And', 'we', 'saw', 'hi', 'm', '!', 'The', 'Cat', 'in', 'the', 'Hat', '!', 'And', 'he', 'sa', 'id', 'to', 'u', 's', '!', '!', '!', 'W', 'h', 'y', 'do', 'you', 's', 'it', 'ther', 'e', 'l', 'i', 'ke', 'that', '?', '!', '!', '!', 'I', 'know', 'it', 'is', 'we', 't', 'And', 'the', 'sun', 'is', 'not', 'sun', 'ny', '!', 'B', 'u', 't', 'we', 'c', 'a', 'n', 'h', 'a', 'v', 'e', 'L', 'ot', 's', 'of', 'good', 'fun', 'that', 'is', 'fun', 'ny', '!', '!', '!', '!', 'I', 'know', 'some', 'good', 'g', 'a', 'me', 's', 'we', 'c', 'ou', 'l', 'd', 'p', 'l', 'ay', '!', '!', '!', 'Said', 'the', 'c', 'at', '!', '!', '!', '!', 'I', 'know', 'some', 'n', 'e', 'w', 'trick', 's', '!', '!', '!', 'Said', 'the', 'Cat', 'in', 'the', 'Hat', '!', '!', '!', '!', 'A', 'l', 'ot', 'of', 'good', 'trick', 's', '!', '!', '!', 'I', 'w', 'ill', 's', 'h', 'o', 'w', 'the', 'm', 'to', 'you', '!', '!', '!', 'Y', 'ou', 'r', 'M', 'other', 'W', 'ill', 'not', 'm', 'i', 'nd', 'at', 'a', 'll', 'i', 'f', 'I', 'do', '!', '!', '!', 'Then', 'Sa', 'll', 'y', 'a', 'nd', 'I', 'D', 'id', 'not', 'know', 'w', 'hat', 'to', 'sa', 'y', '!', '!', '!', 'O', 'u', 'r', 'm', 'other', 'w', 'a', 's', 'ou', 't', 'of', 'the', 'h', 'ou', 's', 'e', 'F', 'o', 'r', 'the', 'd', 'ay', '!', '!']

Vocabulary size = 150

[["We', 'looked', '!', 'Then', 'we', 'saw', 'him', 'st', 'ep', 'in', 'on', 'the', 'mat', '!', 'We', 'looked', '!', 'And', 'we', 'saw', 'him', '!', 'The', 'Cat', 'in', 'the', 'Hat', '!', 'And', 'he', 'sa', 'id', 'to', 'us', '!', '!', '!', '!', 'Wh', 'y', 'do', 'you', 'sit', 'ther', 'e', 'like', 'that', '?', '!', '!', '!', 'I', 'know', 'it', 'is', 'we', 't', 'And', 'the', 'sun', 'is', 'not', 'sun', 'ny', '!', '!', '!', 'Bu', 't', 'we', 'can', 'hav', 'e', 'Lot', 's', 'of', 'good', 'fun', 'that', 'is', 'fun', 'ny', '!', '!', '!', '!', 'I', 'know', 'some', 'good', 'game', 's', 'we', 'cou', 'ld', 'play', '!', '!', '!', 'Said', 'the', 'cat', '!', '!', '!', '!', 'I', 'know', 'some', 'new', 'tricks', '!', '!', '!', 'Said', 'the', 'Cat', 'in', 'the', 'Hat', '!', '!', '!', '!', 'A', 'lot', 'of', 'good', 'tricks', '!', '!', '!', '!', 'I', 'w', 'ill', 'sho', 'w', 'the', 'm', 'to', 'you', '!', '!', '!', 'You', 'r', 'Mother', 'Will', 'not', 'mind', 'at', 'all', 'if', 'I', 'do', '!', '!', '!', 'Then', 'Sa', 'll', 'y', 'and', 'I', 'Did', 'not', 'know', 'w', 'hat', 'to', 'sa', 'y', '!', '!', '!', 'Ou', 'r', 'mother', 'w', 'as', 'ou', 't', 'of', 'the', 'hou', 'se', 'Fo', 'r', 'the', 'day', '!', '!']]